

3.2 FDCAN 例程详细说明

1. 介绍

这是一个用于控制高擎机电电机的软件接口包。

通过这个接口包，用户可以方便地与高擎机电的电机进行通信，获取电机的状态信息，并控制电机的动作。

2. 软件结构介绍

- **App: 硬件**

- `led`：负责 LED 灯的控制。
- `my_fdcan`：与 FDCAN 硬件相关的功能实现。

- **Src: 应用**

- `convert`：负责单位转换，包括电机的位置、速度、加速度、力矩、电压、电流、PID 以及转 (rev)、弧度 (rad)、度 (°) 等单位的转换。
- `livelybot_fdcan`：实现电机底层协议，处理原始数据。
- `motor_control`：用于单电机控制，支持 16 种电机控制模式。
- `motor_many`：用于一拖多模式的电机控制，支持 12 种电机控制模式。
- `motor_config`：负责电机设置相关功能，包括带反馈的电机零位重置和电机设置保存。
- `motor`：包含 FDCAN 接口通道映射、电机类型定义以及电机返回信息解析（包括 FDCAN 中断函数 `HAL_FDCAN_RxFifo0Callback` 的实现），**移植和使用的時候主要就是修改这个部分**。

- **test: 测试**

- `test_motor`：包含单电机函数使用例程和一些简单的控制。
- `test_motor_many`：包含一拖多模式的函数使用例程和一些简单的控制。

3. 移植和配置说明

- 如果没有移植需求可直接跳到 3.2 配置 部分。

- 操作如下

- 将 `App` 文件夹下的 `my_fdcan` 文件夹和 `Src` 文件夹下的 `convert`、`livelybot_fdcan`、`motor_control`、`motor_many`、`motor_config` 和 `motor` 5 个文件夹复制到新工程。

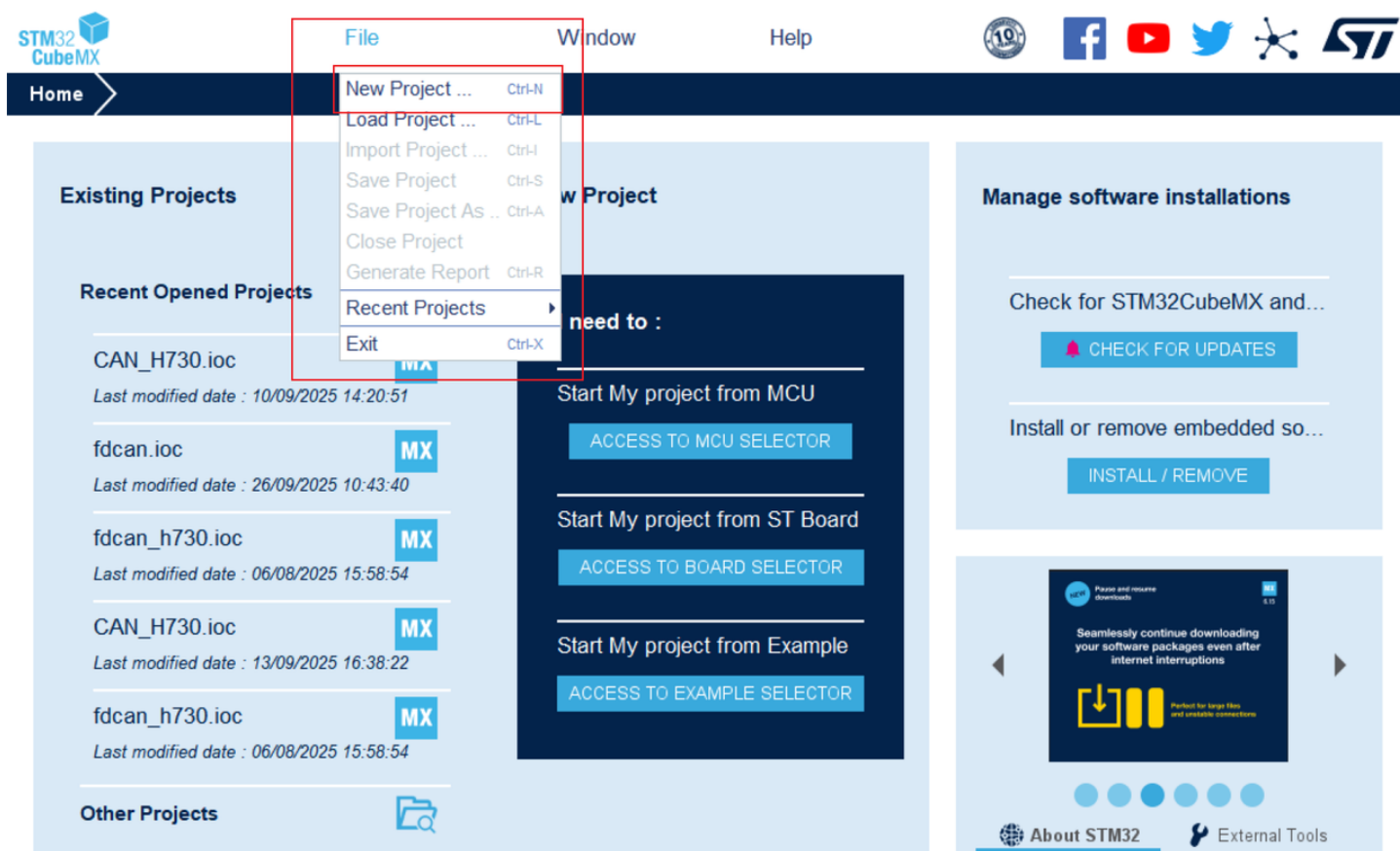
- 将 `my_fdcan`、`convert`、`livelybot_fdcan`、`motor_control`、`motor_many`、`motor_config` 和 `motor` 6 个文件夹都加入到头文件路径。

3.1 移植指南

具体移植操作如下

3.1.1 新建工程

1. 创建项目：打开 STM32CubeMX，点击 `New Project`。
2. 选择芯片：在芯片选择器中，输入您使用的单片机型号（例程使用 STM32H730VBT6），双击选中该型号。
3. 关键系统配置：在 `Pinout & Configuration` 选项卡中，进入 `System Core` -> `SYS` 菜单：
 - 将 `Debug` 选项设置为 `Serial Wire`。此处是为了通过 ST-Link 等进行调试和烧录。



MCU/MPU Selector Board Selector Example Selector Cross Selector

MCU/MPU Filters

Commercial Part Number: STM32H730VBT6

PRODUCT INFO

- Segment
- Series
- Line
- Marketing Status
- Price
- Package
- Core
- Coprocessor

MEMORY

- Flash = 128 (kBytes)
- EEPROM = 0 (Bytes)

STM32H7 Series

STM32H730VBT6

High-performance and DSP with DP-FPU, Arm Cortex-M7 MCU with 128 Kbytes Flash, 564 Kbytes RAM, 550 MHz CPU, L1 cache, external memory interface, subset of peripherals including a Crypto accelerator

Unit Price for 10kU (US\$): 4.5658

LQFP 100 14x14x1.4 mm

STM32H730xB devices are based on the high-performance Arm® Cortex®-M7 32-bit RISC core operating at up to 550 MHz. The Cortex®-M7 core features a floating-point unit (FPU) which supports Arm® double-precision (IEEE 754

MCUs/MPUs List: 1 item

Commercial...	Part No	Reference	Marketin...	Unit Price for ...	Bo...	Package	Flash	RAM	I/O	Frequ...
★	STM32H730V...	STM32H7...	STM32H730...	Active	4.5658	LQFP 100 14x14x1.4 ...	128 kB...	564 kB...	82	550 M...

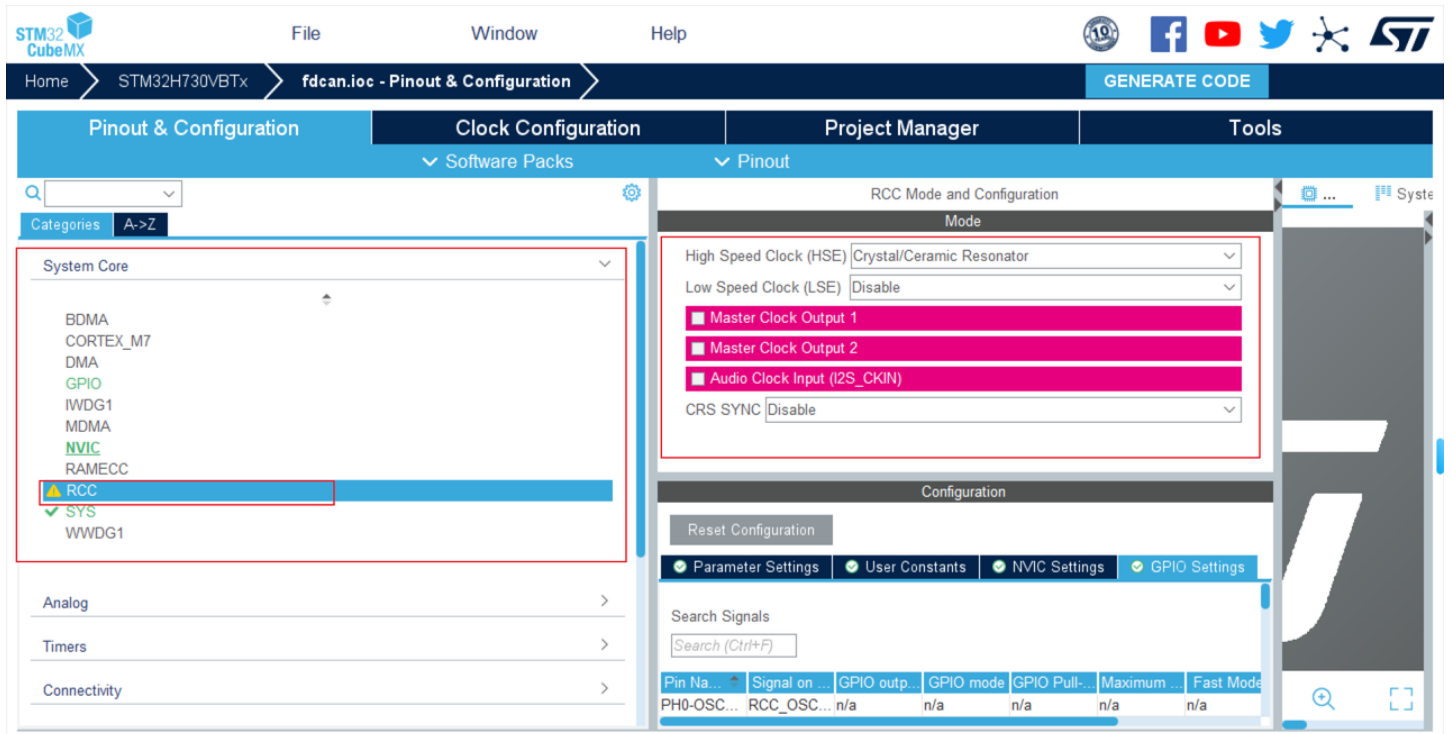
3.1.2 时钟与 FDCAN 配置

本部分是移植成功的关键，请严格按照步骤操作。

3.1.2.1 配置外部高速时钟 (HSE)

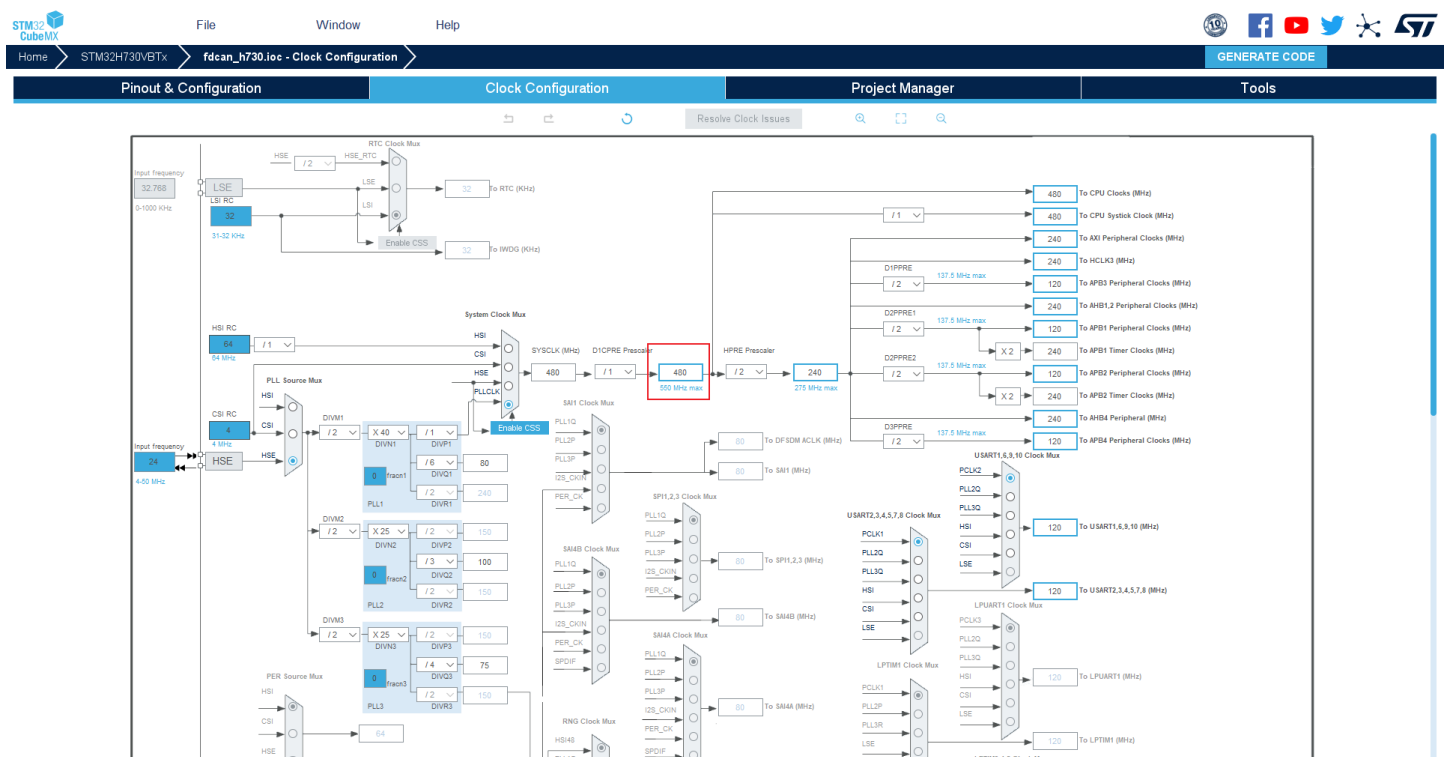
为了确保系统时钟准确稳定，需要配置外部高速时钟源。

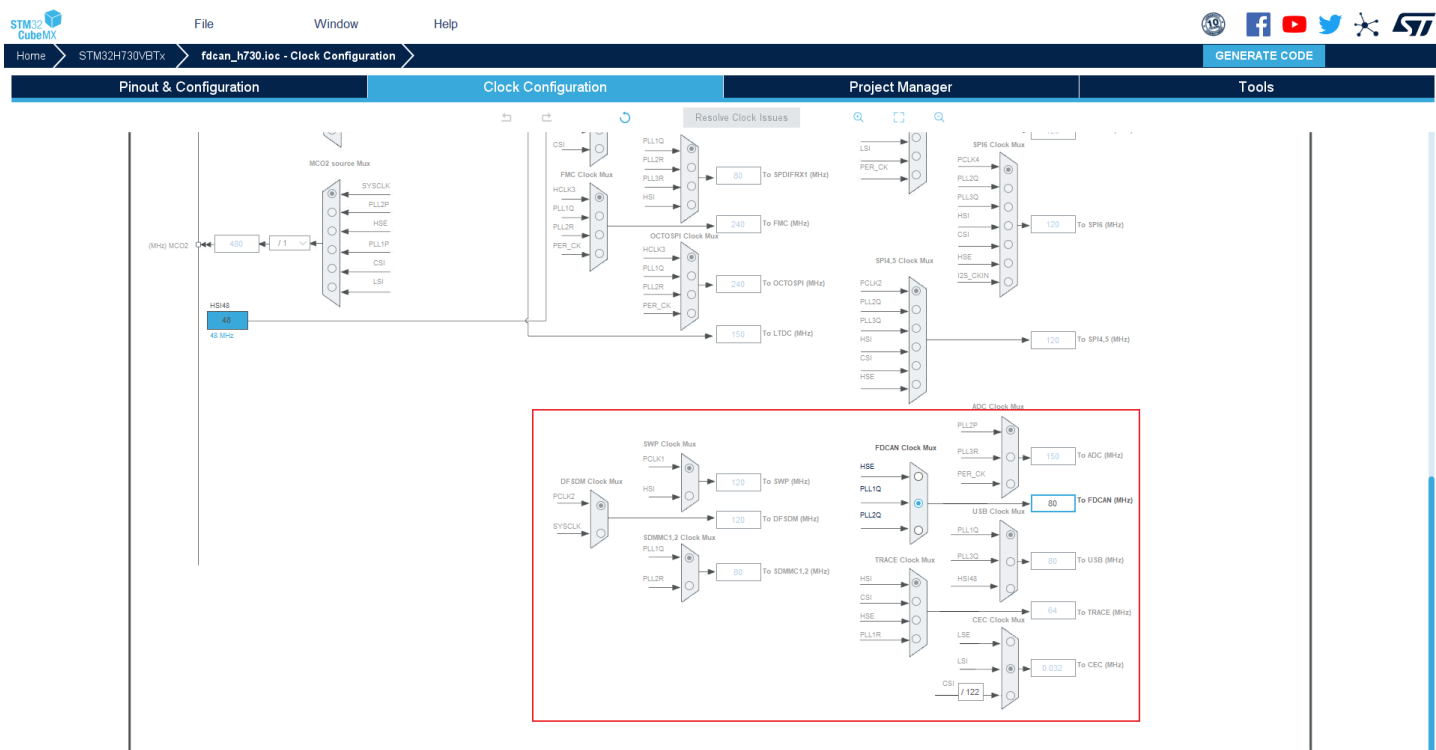
1. 在左侧分类中找到 System Core > RCC（复位和时钟控制）。
2. 在右侧的 RCC Mode and Configuration 模式下，找到 High Speed Clock (HSE) 选项。
3. 将其从默认的 "Disable" 修改为 Crystal/Ceramic Resonator。
 - 作用：这告诉单片机，我们将使用外接的晶振作为高速时钟源，而不是使用芯片内部的 RC 振荡器。外部晶振能提供更精确的时钟频率，是整个系统（包括 FDCAN 通信波特率）稳定的基础。



3.1.2.2 配置时钟树

1. 进入 **Clock Configuration** 选项卡。
2. 根据芯片情况设置主频率，此处配置为 480MHz。
3. 找到并确认 FDCAN 的时钟源频率为 80MHz。此频率是计算 FDCAN 通信波特率的基础，必须相同。





3.1.2.3 配置 FDCAN

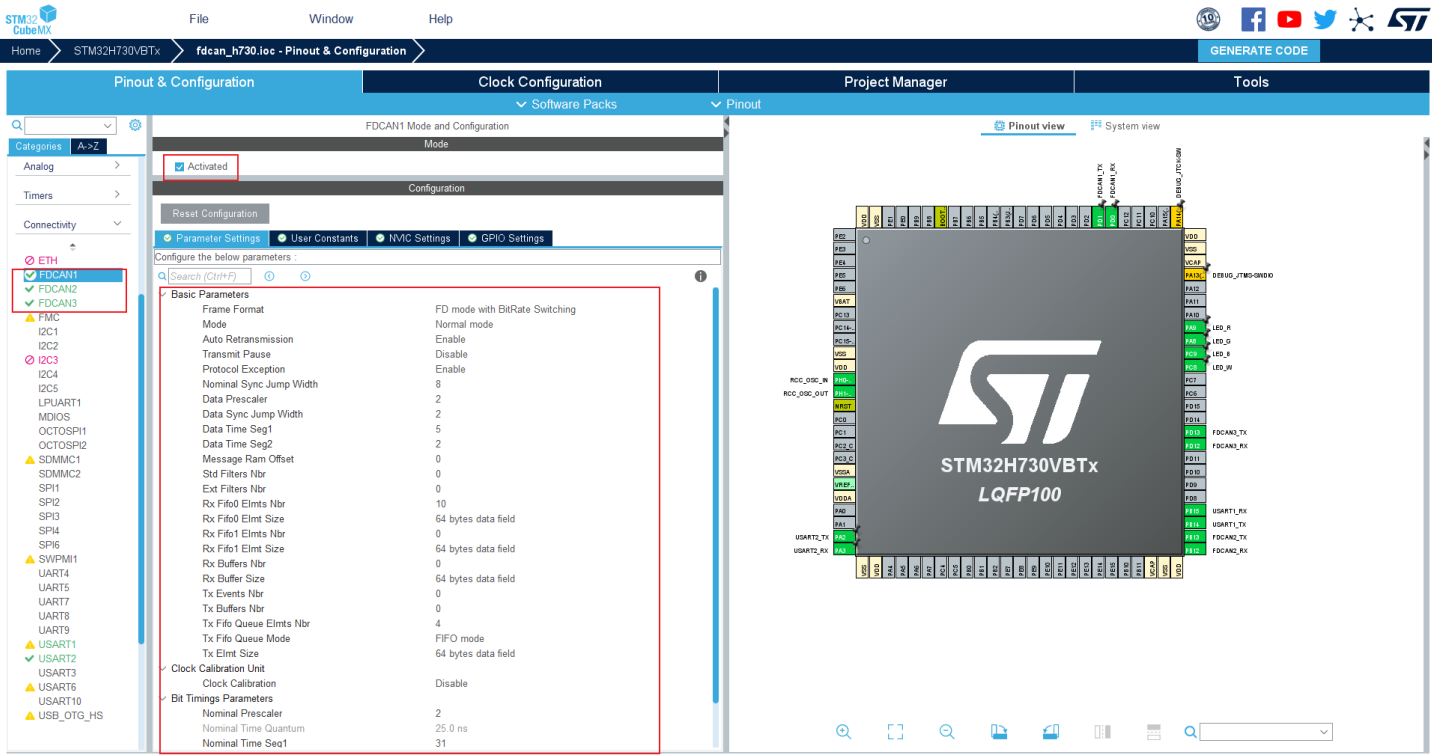
1. 在 **Connectivity** 菜单下，启用您计划使用的所有 FDCAN 模块（如 FDCAN1, FDCAN2, FDCAN3）。
2. 对每个启用的 FDCAN 模块，根据下表进行参数配置（此配置基于 80MHz 时钟，可实现仲裁段 1Mbps，数据段 5Mbps）：

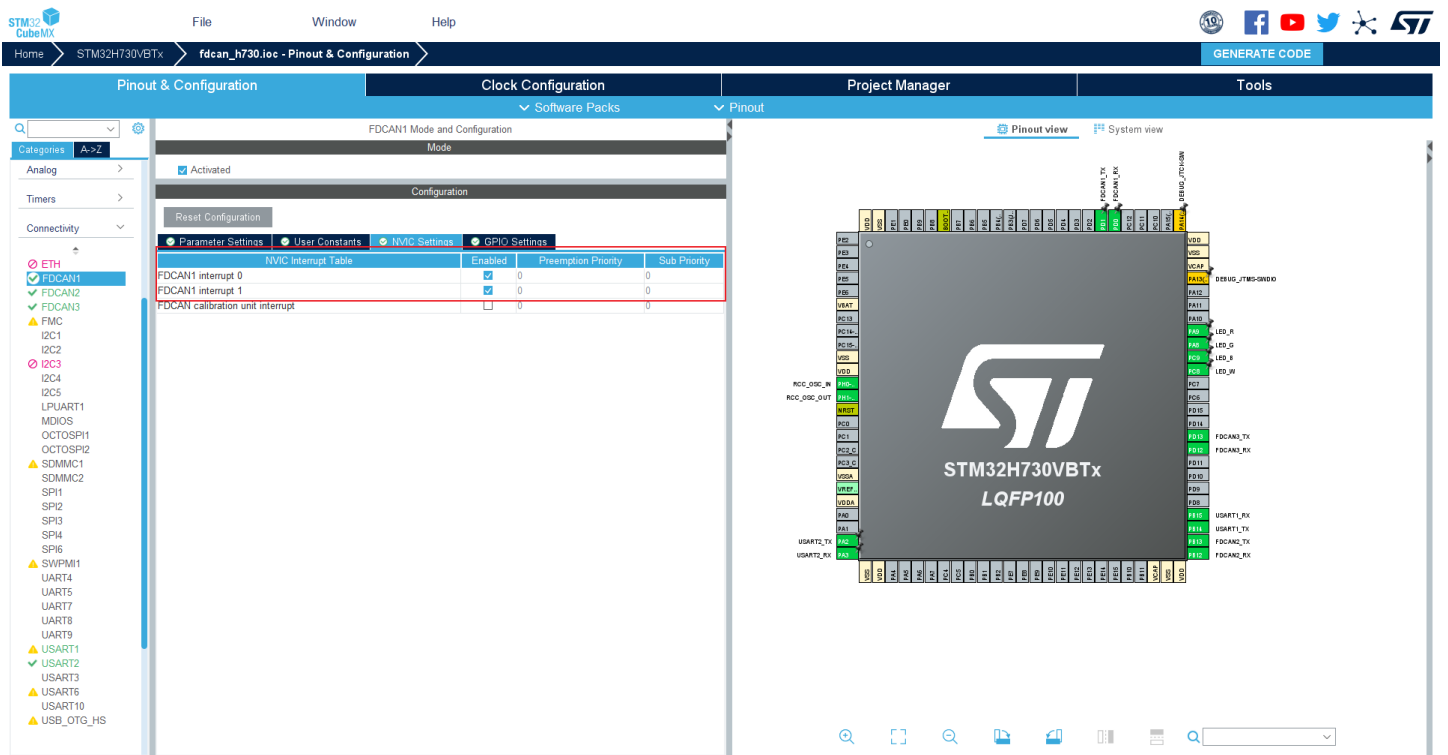
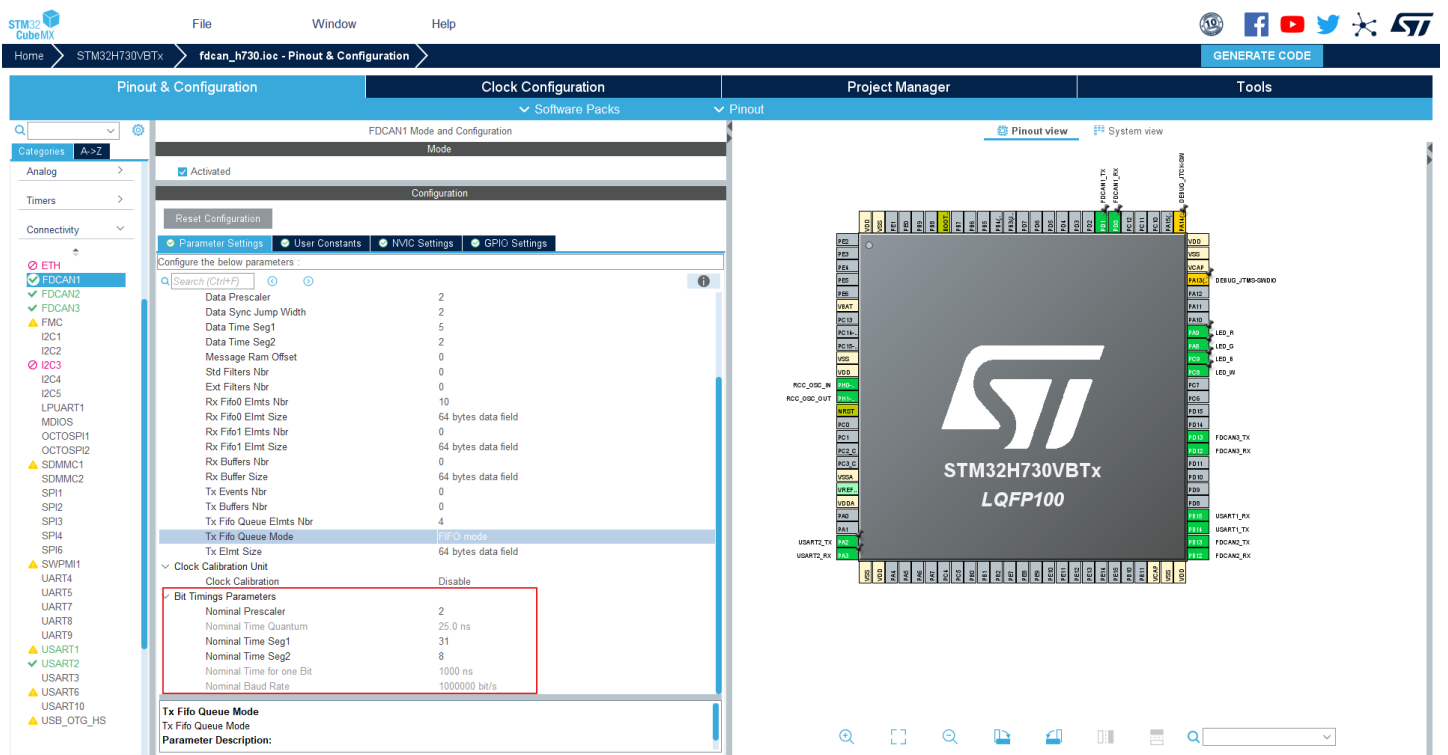
参数分类	参数名称	设定值	说明
	Frame Format	FD mode with BitRate Switching	启用比特率切换
	Mode	Normal mode	模式：Normal / FD
	Auto Retransmission	Enable	自动重传
	Transmit Pause	Disable	发送暂停
	Protocol Exception	Enable	协议异常处理
	Nominal Sync Jump Width	8	仲裁段同步跳转宽度
	Data Prescaler	2	数据段预分频
	Data Sync Jump Width	2	数据段同步跳转宽度

基本参数	Comp Width		
	Data Time Seq1	5	数据段相位缓冲段1
	Data Time Seq2	2	数据段相位缓冲段2
	Message Ram Offset	0	消息 RAM 的起始地址偏移量
	Std Filters Nbr	0	标准ID过滤器数量
	Ext Filters Nbr	0	扩展ID过滤器数量
	Rx Fifo0 Elmts Nbr	10	Rx FIFO0 元素数量
	Rx Fifo0 Elmt Size	64 bytes	FIFO0 元素大小
	Rx Fifo1 Elmts Nbr	0	Rx FIFO1 元素数量
	Rx Fifo1 Elmt Size	64 bytes	FIFO1 元素大小
	Rx Buffers Nbr	0	Rx 缓冲区数量
	Rx Buffer Size	64 bytes	Rx 缓冲区大小
	Tx Events Nbr	0	Tx 事件数量
	Tx Buffers Nbr	0	Tx 缓冲区数量
	Tx Fifo Queue Elmts Nbr	4	Tx FIFO 队列元素数量
	Tx Fifo Queue Mode	FIFO mode	Tx 队列模式
	Tx Elmt Size	64 bytes	Tx 元素大小
时钟校准	Clock Calibration	Disable	时钟校准开关
	Nominal Prescaler	2	仲裁段预分频
	Nominal Time Quantum	25.0 ns	仲裁段时间量子

位定时参数	Nominal Time Seq1	31	仲裁段相位缓冲段1
	Nominal Time Seq2	8	仲裁段相位缓冲段2
	Nominal Time for one Bit	1000 ns	仲裁段每比特时间
	Nominal Baud Rate	1000000 bit/s	仲裁段波特率

3. 开启中断：对每个 FDCAN 模块，进入 NVIC Settings 标签页，勾选使能 FDCANx Interrupt 0（x 为 CAN 编号）。这是接收电机数据的必要条件，至少要开启两个 FDCAN 通道的中断。





4. 可在程序中的 `fdcan.c` 查看配置情况

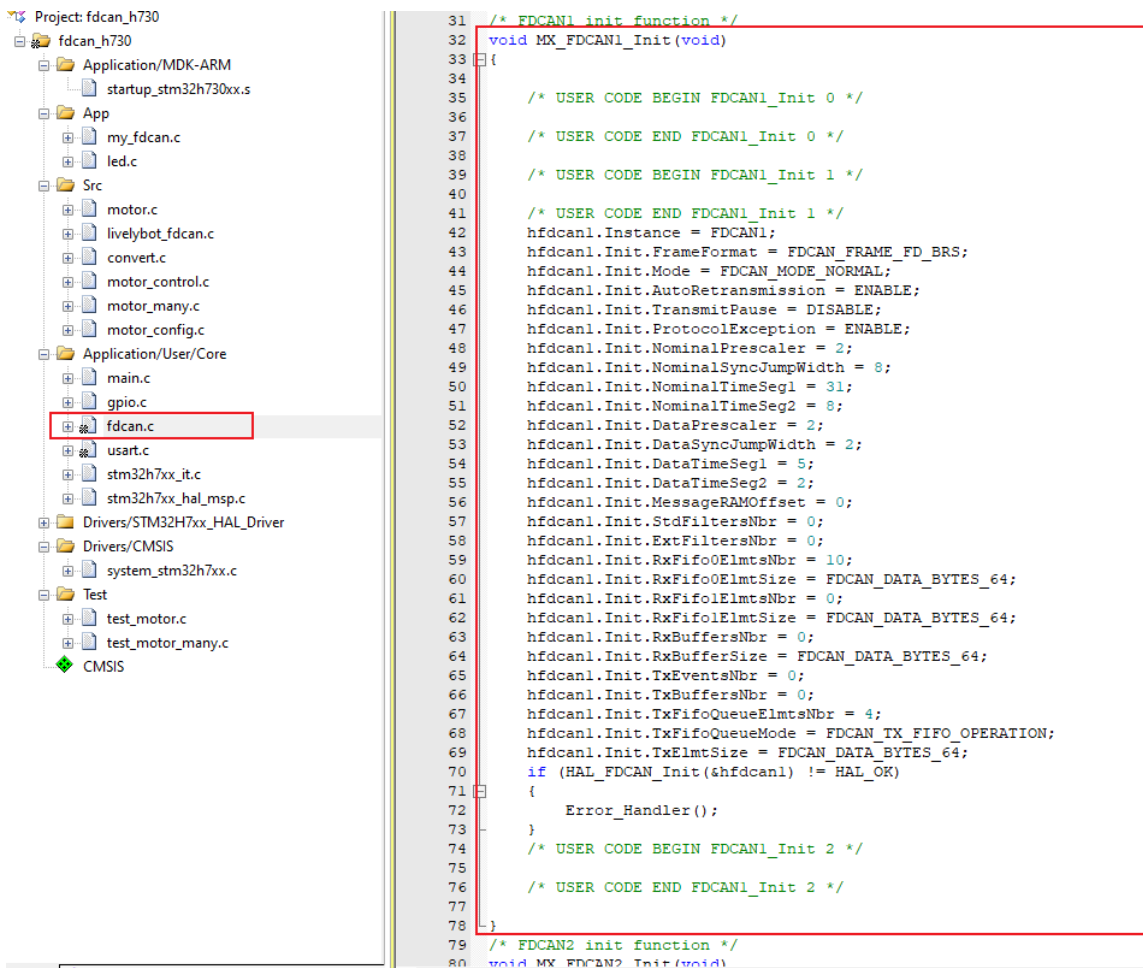
代码块

```
1 hfdcan1.Instance = FDCAN1;
2 hfdcan1.Init.FrameFormat = FDCAN_FRAME_FD_BRS;
3 hfdcan1.Init.Mode = FDCAN_MODE_NORMAL;
4 hfdcan1.Init.AutoRetransmission = ENABLE;
5 hfdcan1.Init.TransmitPause = DISABLE;
6 hfdcan1.Init.ProtocolException = ENABLE;
7 hfdcan1.Init.NominalPrescaler = 2;
```

```

8      hfdcan1.Init.NominalSyncJumpWidth = 8;
9      hfdcan1.Init.NominalTimeSeg1 = 31;
10     hfdcan1.Init.NominalTimeSeg2 = 8;
11     hfdcan1.Init.DataPrescaler = 2;
12     hfdcan1.Init.DataSyncJumpWidth = 2;
13     hfdcan1.Init.DataTimeSeg1 = 5;
14     hfdcan1.Init.DataTimeSeg2 = 2;
15     hfdcan1.Init.MessageRAMOffset = 0;
16     hfdcan1.Init.StdFiltersNbr = 0;
17     hfdcan1.Init.ExtFiltersNbr = 0;
18     hfdcan1.Init.RxFifo0ElmtsNbr = 10;
19     hfdcan1.Init.RxFifo0ElmtSize = FDCAN_DATA_BYTES_64;
20     hfdcan1.Init.RxFifo1ElmtsNbr = 0;
21     hfdcan1.Init.RxFifo1ElmtSize = FDCAN_DATA_BYTES_64;
22     hfdcan1.Init.RxBuffersNbr = 0;
23     hfdcan1.Init.RxBufferSize = FDCAN_DATA_BYTES_64;
24     hfdcan1.Init.TxEventsNbr = 0;
25     hfdcan1.Init.TxBuffersNbr = 0;
26     hfdcan1.Init.TxFifoQueueElmtsNbr = 4;
27     hfdcan1.Init.TxFifoQueueMode = FDCAN_TX_FIFO_OPERATION;
28     hfdcan1.Init.TxElmtSize = FDCAN_DATA_BYTES_64;

```



```

31  /* FDCAN1 init function */
32  void MX_FDCAN1_Init(void)
33  {
34
35      /* USER CODE BEGIN FDCAN1_Init 0 */
36
37      /* USER CODE END FDCAN1_Init 0 */
38
39      /* USER CODE BEGIN FDCAN1_Init 1 */
40
41      /* USER CODE END FDCAN1_Init 1 */
42      hfdcan1.Instance = FDCAN1;
43      hfdcan1.Init.FrameFormat = FDCAN_FRAME_FD_BRS;
44      hfdcan1.Init.Mode = FDCAN_MODE_NORMAL;
45      hfdcan1.Init.AutoRetransmission = ENABLE;
46      hfdcan1.Init.TransmitPause = DISABLE;
47      hfdcan1.Init.ProtocolException = ENABLE;
48      hfdcan1.Init.NominalPrescaler = 2;
49      hfdcan1.Init.NominalSyncJumpWidth = 8;
50      hfdcan1.Init.NominalTimeSeg1 = 31;
51      hfdcan1.Init.NominalTimeSeg2 = 8;
52      hfdcan1.Init.DataPrescaler = 2;
53      hfdcan1.Init.DataSyncJumpWidth = 2;
54      hfdcan1.Init.DataTimeSeg1 = 5;
55      hfdcan1.Init.DataTimeSeg2 = 2;
56      hfdcan1.Init.MessageRAMOffset = 0;
57      hfdcan1.Init.StdFiltersNbr = 0;
58      hfdcan1.Init.ExtFiltersNbr = 0;
59      hfdcan1.Init.RxFifo0ElmtsNbr = 10;
60      hfdcan1.Init.RxFifo0ElmtSize = FDCAN_DATA_BYTES_64;
61      hfdcan1.Init.RxFifo1ElmtsNbr = 0;
62      hfdcan1.Init.RxFifo1ElmtSize = FDCAN_DATA_BYTES_64;
63      hfdcan1.Init.RxBuffersNbr = 0;
64      hfdcan1.Init.RxBufferSize = FDCAN_DATA_BYTES_64;
65      hfdcan1.Init.TxEventsNbr = 0;
66      hfdcan1.Init.TxBuffersNbr = 0;
67      hfdcan1.Init.TxFifoQueueElmtsNbr = 4;
68      hfdcan1.Init.TxFifoQueueMode = FDCAN_TX_FIFO_OPERATION;
69      hfdcan1.Init.TxElmtSize = FDCAN_DATA_BYTES_64;
70      if (HAL_FDCAN_Init(&hfdcan1) != HAL_OK)
71      {
72          Error_Handler();
73      }
74      /* USER CODE BEGIN FDCAN1_Init 2 */
75
76      /* USER CODE END FDCAN1_Init 2 */
77
78  }
79
80  /* FDCAN2 init function */
81  void MX_FDCAN2_Init(void)

```

3.1.2.4 配置其他外设（可选）

STM32CubeMX

FileWindowHelp

HomeSTM32H730VBTxfdcan_h730.iocPinout & Configuration

GENERATE CODE

Pinout & Configuration

Clock Configuration

Project Manager

Tools

CategoriesA-Z

System Core

BOMACORTEX_M7DMA
GPIO
IWDG1MDIAMA
NVICRAMECC
RCC
SYS
WWDG1

Analog
ADC1
ADC2
ADC3
COMP1
COMP2
DAC1
DTS
OPAMP1
OPAMP2
VREFBUF

Timers
Connectivity
ETH
FDCAN1
FDCAN2
FDCAN3
FMC
I2C1

Configuration

Show All

GPIO

Search Signals

Show only Modified Pins

Pin Name	Signal o...	GPIO output	GPIO mode	GPIO Pullu...	Maximum o...	Fast Mode	User Label	Modified
PA14(JTCK_	DEBUG_JT...	n/a	n/a	n/a	n/a	n/a		☐
PA13(JTMS...	DEBUG_JT...	n/a	n/a	n/a	n/a	n/a		☐
PD0	FDCAN1_RX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PD1	FDCAN1_TX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PB12	FDCAN2_RX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PB13	FDCAN2_TX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PD12	FDCAN3_RX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PD13	FDCAN3_TX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PH0-OSC_IN	RCC_OSC_...	n/a	n/a	n/a	n/a	n/a		☐
PH1-OSC_...	RCC_OSC_...	n/a	n/a	n/a	n/a	n/a		☐
PB15	USART1_RX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PB14	USART1_TX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PA3	USART2_RX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PA2	USART2_TX	n/a	Alternate F...	No pull-up ...	Low	n/a		☐
PA8	n/a	High	Output Pus...	No pull-up ...	Low	n/a	LED_G	☒

PA8 Configuration :

GPIO output level
High

GPIO mode
Output Push Pull

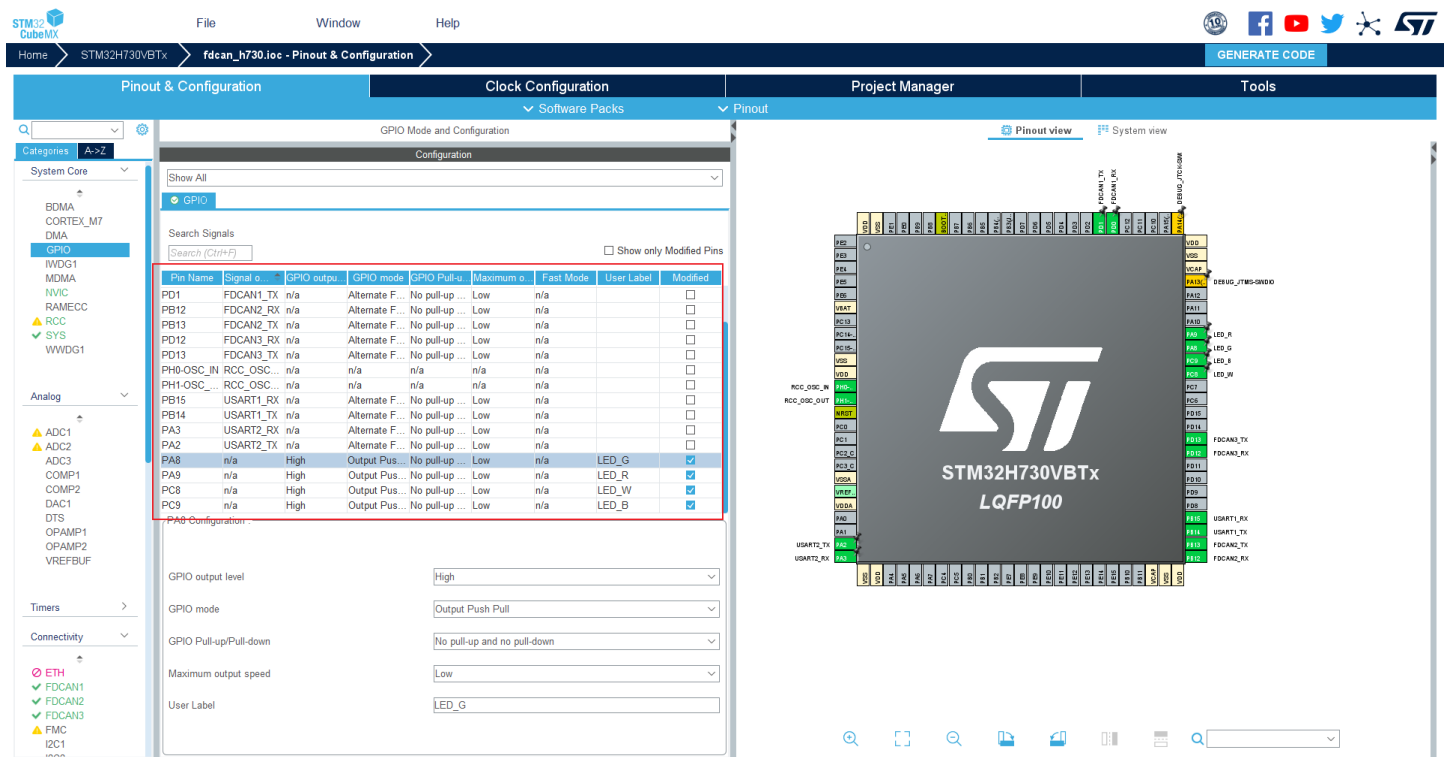
GPIO Pull-up/Pull-down
No pull-up and no pull-down

Maximum output speed
Low

User Label
LED_G

Pinout view

System view



3.1.3.1 生成代码：

- 进入 **Project Manager** -> **Code Generator** 页面进行关键设置:

- 取消勾选 `Generate peripheral initialization as a pair of '.c/.h' files per peripheral`。此设置能将所有外设初始化代码整合到 `main.c` 中，极大简化项目结构，避免产生过多零散文件。
- 务必勾选 `Keep User Code when re-generating`。这是最重要的设置，可以保护您在特定注释标签（`/* USER CODE BEGIN */`）内编写的代码在重新生成时不被覆盖。
- 进入 `Project Manager` -> `Project` 选项卡，为项目命名并选择保存路径。
 - 在 `Toolchain / IDE` 选项中选择您使用的 IDE（如 `MDK-ARM`）。

2. 生成项目文件

完成设置后，回到 `Project` 页面，检查项目路径和 IDE 选项无误，点击 `GENERATE CODE` 按钮生成完整的工程代码。

The screenshot displays the STM32CubeMX Project Manager interface. The top navigation bar includes 'File', 'Window', and 'Help' menus, along with social media icons and the ST logo. The main window is titled 'fdcan.ioc - Project Manager' and features a 'GENERATE CODE' button. The left sidebar shows four tabs: 'Pinout & Configuration', 'Clock Configuration', 'Project Manager' (selected), and 'Tools'. The 'Project Manager' tab is active, showing the 'Project Settings' section. This section includes fields for 'Project Name' (fdcan), 'Project Location' (F:\1\fdcan), 'Application Structure' (Advanced), 'Toolchain Folder Location' (F:\1\fdcan\fdcan\), and 'Toolchain / IDE' (MDK-ARM). There are also checkboxes for 'Do not generate the main()' and 'Generate Under Root'. Below this, the 'Linker Settings' section shows 'Minimum Heap Size' (0x200) and 'Minimum Stack Size' (0x400). The 'Thread-safe Settings' section includes 'Cortex-M7NS' and 'Enable multi-threaded support'. The 'Thread-safe Locking Strategy' is set to 'Default - Mapping suitable strategy depending on RTOS selection'. The 'Mcu and Firmware Package' section shows 'Mcu Reference' (STM32H730VBTx) and 'Firmware Package Name and Version' (STM32Cube FW_H7 V1.11.2). There are checkboxes for 'Use latest available version' and 'Use Default Firmware Location'.

Pinout & Configuration	Clock Configuration	Project Manager	Tools
Project	STM32Cube MCU packages and embedded software packs		
Code Generator	☐ Copy all used libraries into the project folder ☒ Copy only the necessary library files ☐ Add necessary library files as reference in the toolchain project configuration file		
Advanced Settings	Generated files ☒ Generate peripheral initialization as a pair of '.c/.h' files per peripheral ☐ Backup previously generated files when re-generating ☒ Keep User Code when re-generating ☒ Delete previously generated files when not re-generated		
	HAL Settings ☐ Set all free pins as analog (to optimize the power consumption) ☐ Enable Full Assert		
	Template Settings Select a template to generate customized code Settings...		

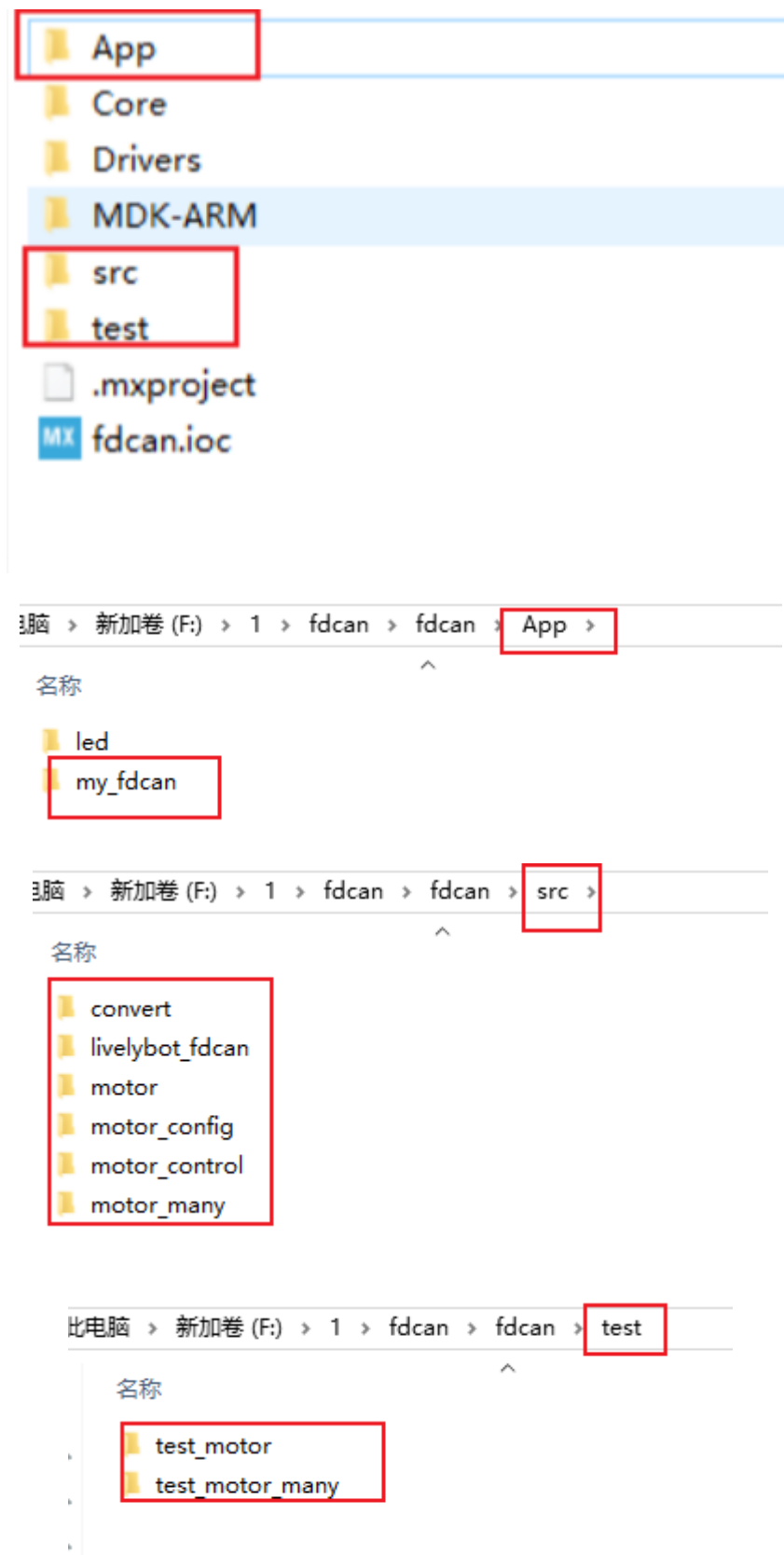
3.1.3.2 文件移植

- 将以下文件夹从原始软件包目录完整复制到您新生成的 STM32 工程根目录下：

代码块

```
1  |— App/
2  |   |— my_fdcan/           # 硬件抽象层 (HAL)
3  |— Src/
4  |   |— convert/           # 单位转换
5  |   |— livelybot_fdcan/   # 底层协议处理
6  |   |— motor_control/     # 单电机控制
7  |   |— motor_many/        # 一拖多控制
8  |   |— motor_config/      # 电机设置 (如零位重置)
9  |   |— motor/             # 【核心】电机类型定义、通信映射、数据解析
10 |   |— test/              # (建议复制, 供参考)
11 |   |— test_motor/        # 单电机使用例程
12 |   |— test_motor_many/   # 一拖多使用例程
```

- (建议复制) 将 `test/test_motor` 和 `test/test_motor_many` 文件夹也复制到工程中，作为使用范例参考。



3.1.3.3 添加头文件路径：

- 在 IDE（如 Keil MDK）中打开工程，进入 Options for Target -> C/C++ 选项卡。
- 在 Include Paths 中，添加以下所有路径（请仔细核对，确保路径正确）：

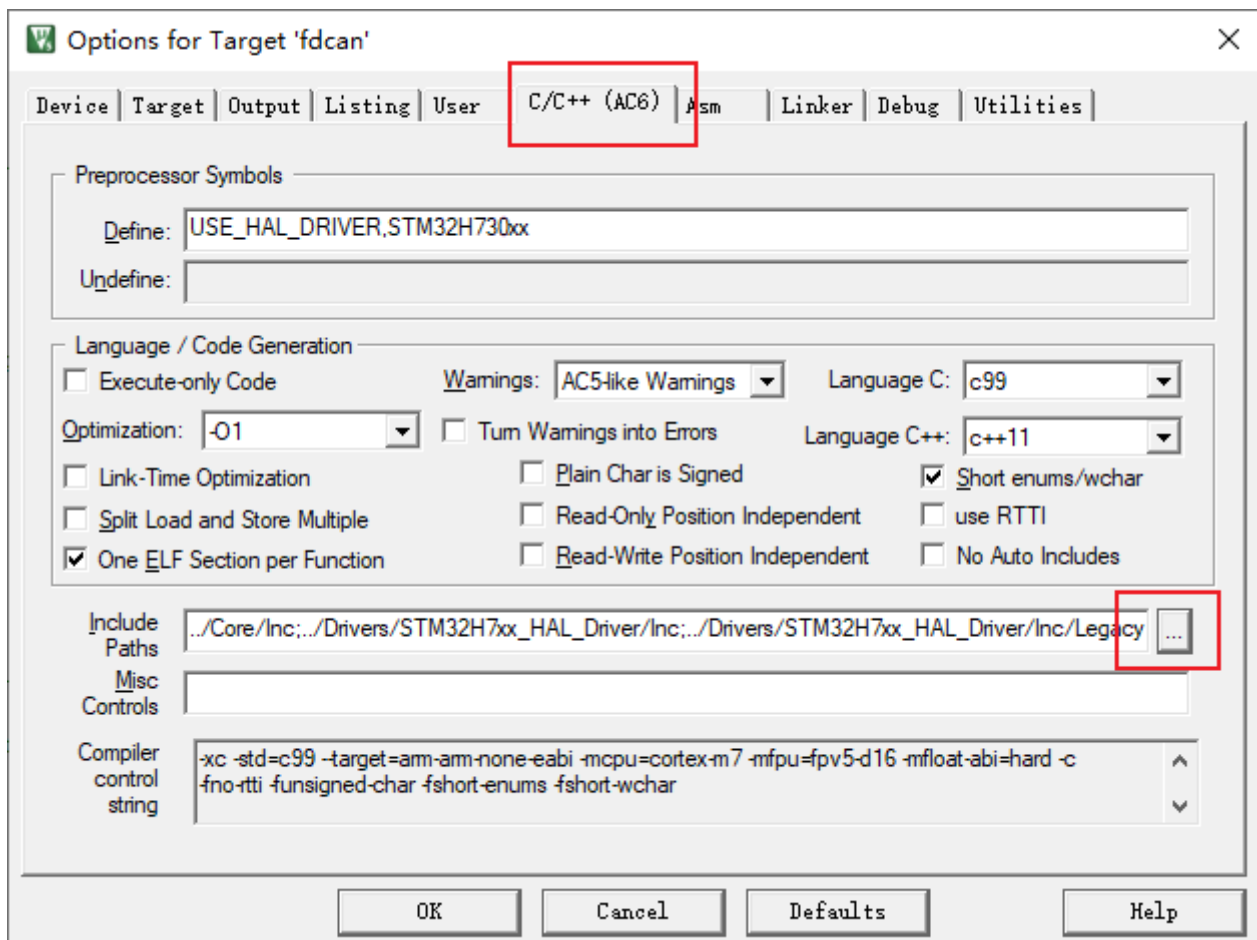
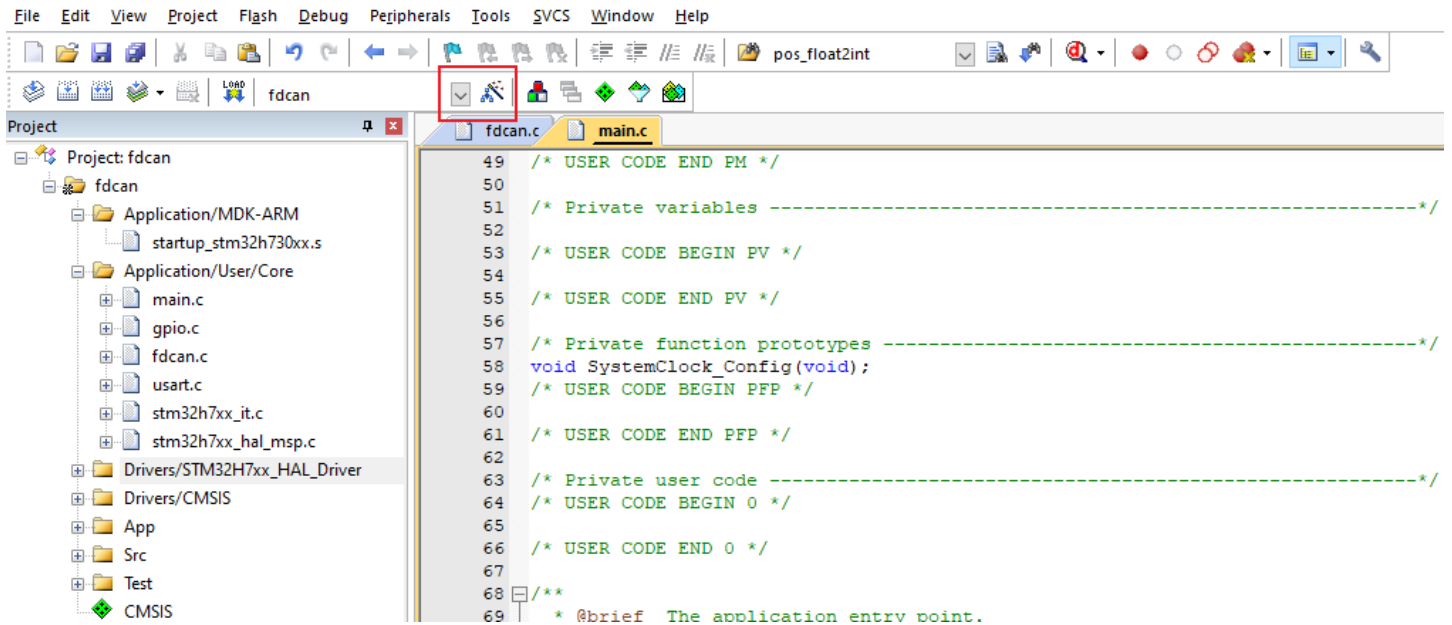
代码块

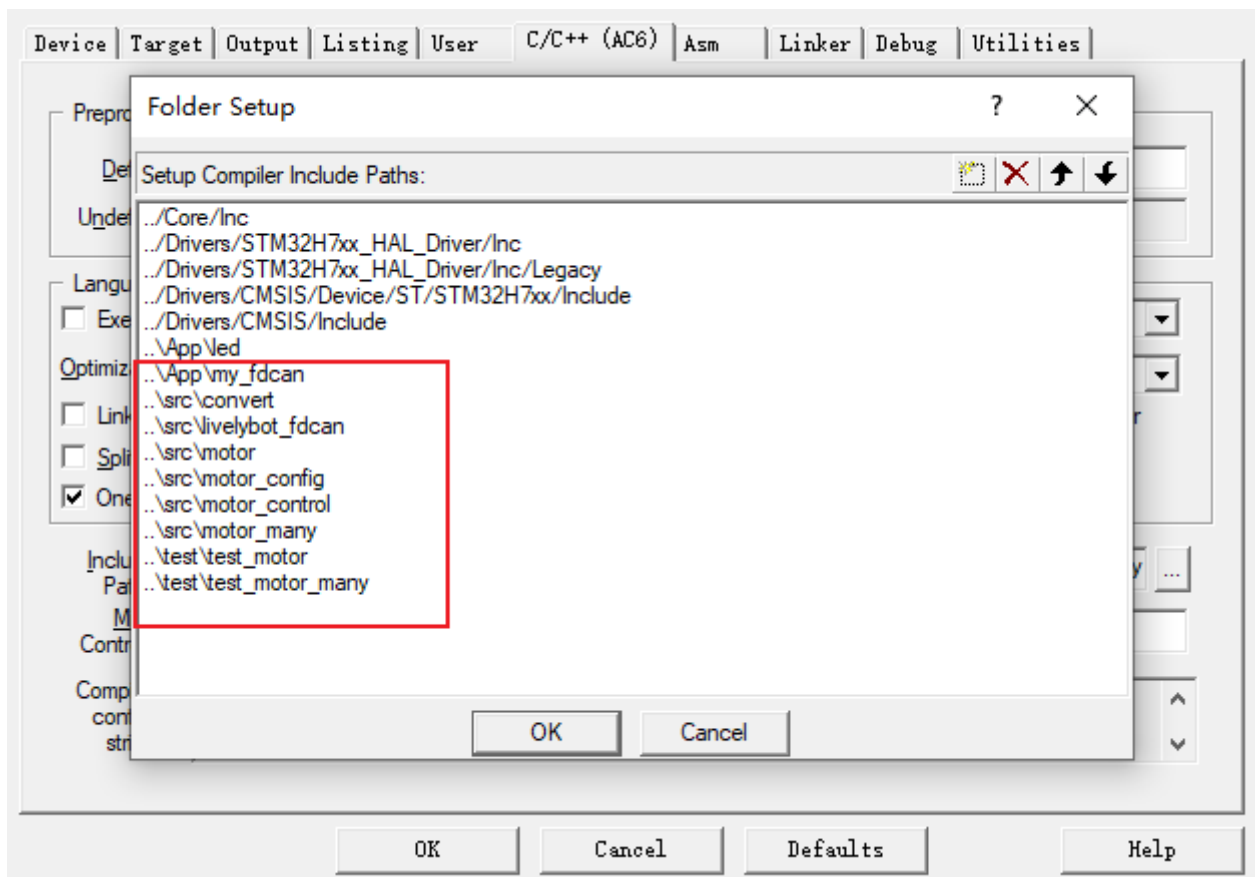
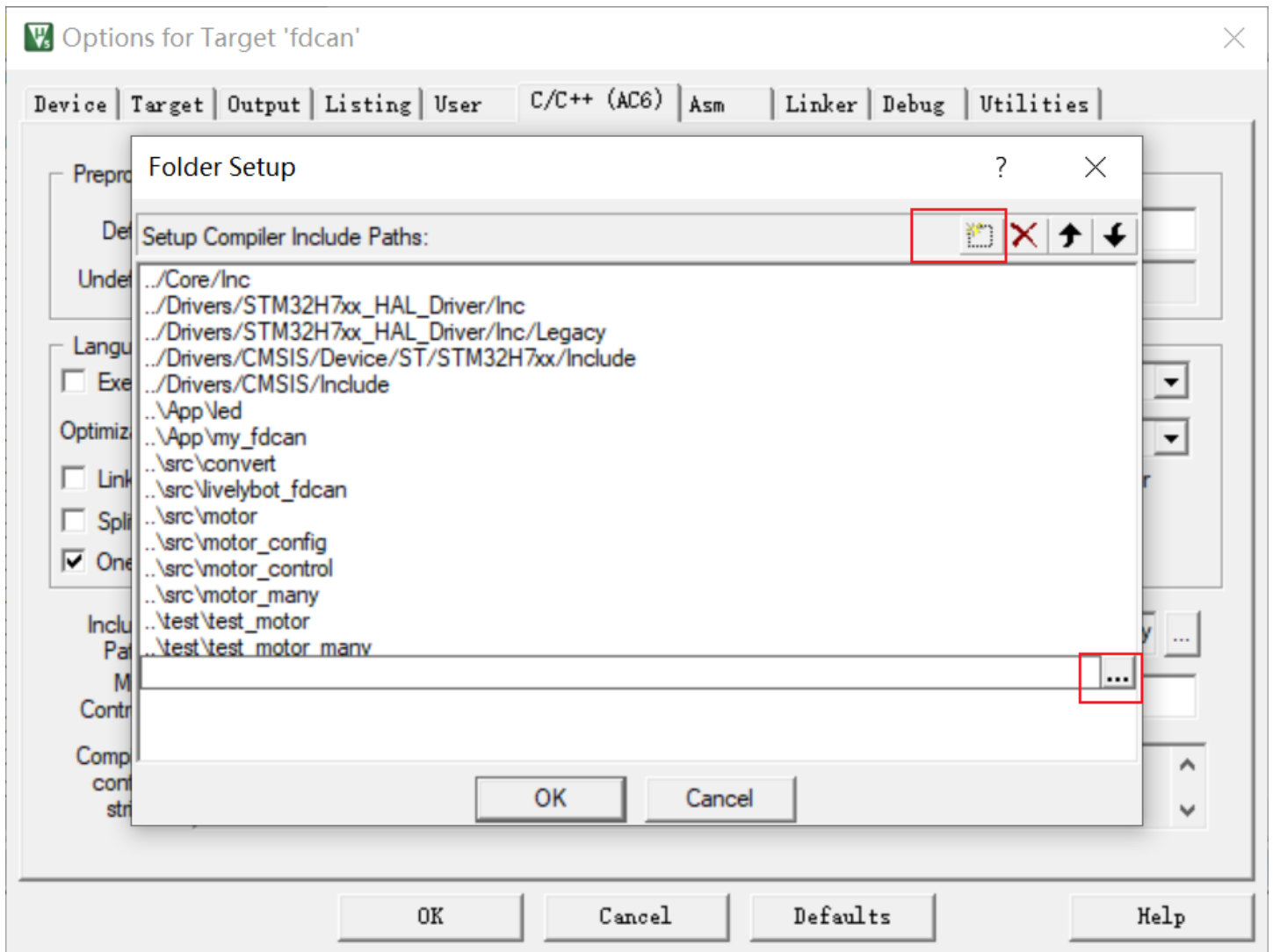
```
1 .\App\my_fdcan
2 .\Src\convert
```

```

3  .\Src\livelybot_fdcan
4  .\Src\motor_config
5  .\Src\motor_control
6  .\Src\motor_many
7  .\Src\motor
8  .\test\test_motor
9  .\test\test_motor_many

```



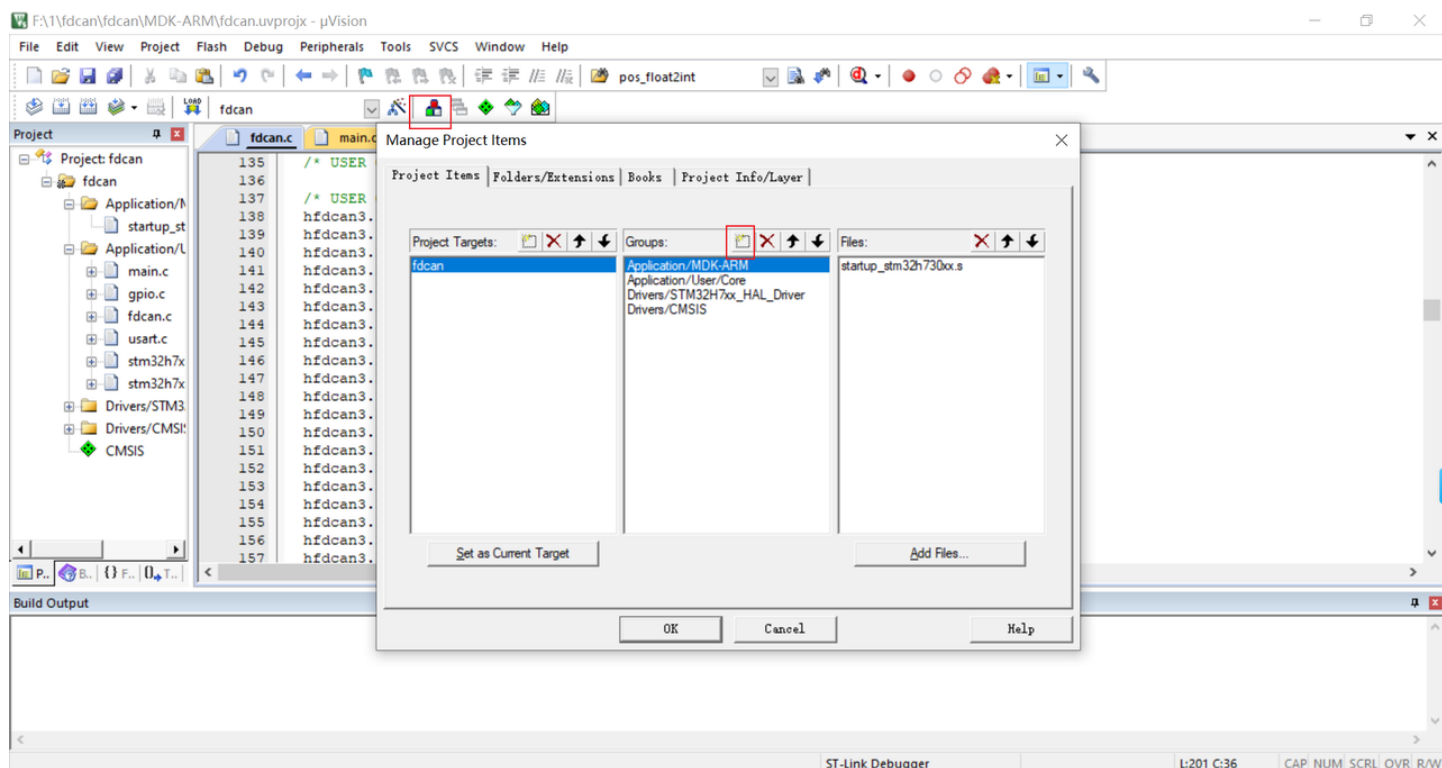


3.1.3.4 添加源文件到工程

1. 在 IDE 的项目管理窗口中，创建三个新的文件组（Group），分别命名为 `App`，`Src`，和 `Test`。
2. 将对应的源文件（`.c` 文件）拖放或添加到相应的组中：
 - `App` 组：添加 `my_fdcan.c`、`led.c` (不是必要的)。
 - `Src` 组：添加 `motor.c`，`livelybot_fdcan.c`，`convert.c`，`motor_control.c`，`motor_many.c`，`motor_config.c`。
 - `Test` 组（可选）：添加 `test_motor.c`，`test_motor_many.c` 作为参考例程。
3. 将对应的源文件添加到各自的组中。添加成功后，您可以在 IDE 的项目管理器界面中清晰地看到完整的、按功能模块分组的工程文件结构。

代码块

```
1  my_fdcan.c
2  convert.c
3  livelybot_fdcan.c
4  motor_control.c
5  motor_many.c
6  motor_config.c
7  motor.c
8  （可选）test_motor.c 和 test_motor_many.c
```



Manage Project Items



Project Items | Folders/Extensions | Books | Project Info/Layer

Project Targets:    

fdcan

Set as Current Target

Groups:    

Application/MDK-ARM
Application/User/Core
Drivers/STM32H7xx_HAL_Driver
Drivers/CMSIS
App
Src
Test

Files:   

led.c
my_fdcan.c

Add Files...

OK

Cancel

Help

Project Targets:    

fdcan

Groups:    

Application/MDK-ARM
Application/User/Core
Drivers/STM32H7xx_HAL_Driver
Drivers/CMSIS
App
Src
Test

Files:   

convert.c
livelybot_fdcan.c
motor.c
motor_config.c
motor_control.c
motor_many.c

Set as Current Target

Add Files...

OK

Cancel

Help

Manage Project Items



Project Items | Folders/Extensions | Books | Project Info/Layer

Project Targets:    

fdcan

Groups:    

Application/MDK-ARM
Application/User/Core
Drivers/STM32H7xx_HAL_Driver
Drivers/CMSIS
App
Src
Test

Files:   

test_motor_many.c
test_motor.c

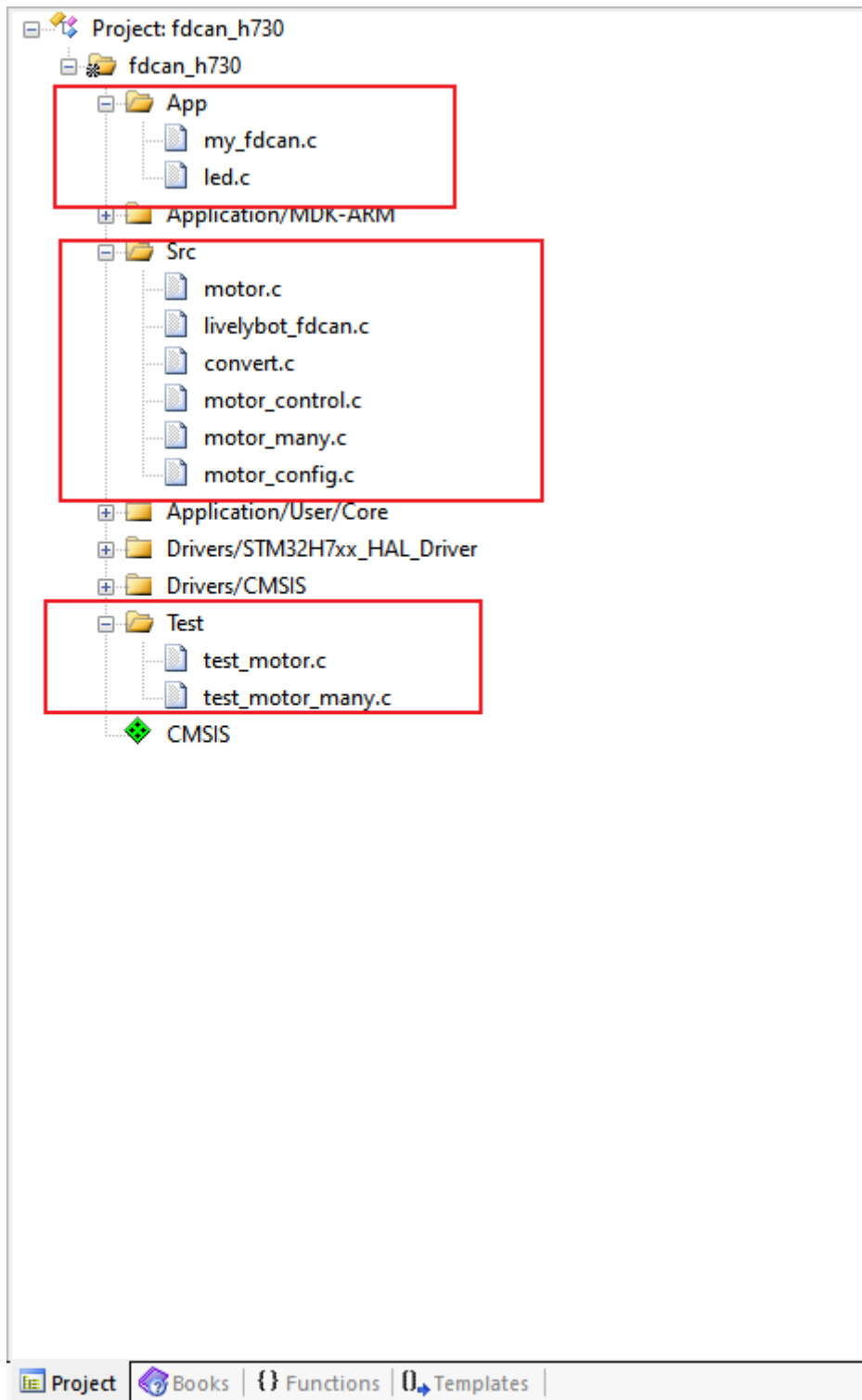
Set as Current Target

Add Files...

OK

Cancel

Help



3.1.3.5 在项目中包含头文件

在您的主程序文件（如 `main.c`）中，包含必要的头文件：

代码块

```
1  #include "motor.h"
2  #include "motor_control.h"
3  #include "motor_many.h"
4  #include "motor_config.h"
5
6  #include "test_motor.h"
7  #include "test_motor_many.h"
```

3.1.3.6 修改报错宏：

1. 在 `convert.h` 文件中，有用于检查某些配置或使用错误的宏，默认使用 `led_toggle_err` 所有 LED 同时闪烁报错，可根据需要自行修改。
2. `#include "led.h"` 这里引入 led 头文件只为使用 `led_toggle_err` 函数，如不需要直接删除即可。

代码块

```
1  #include "led.h"
2  #ifdef LED_ERR_FLAG // 这个宏定义在 led.h 中
3  #define MOTOR_ERR    led_toggle_err // 所有 led 闪烁
4  #else
5  static inline void MOTOR_ERR(void) {}
6  #endif
```

3. 编辑成功，移植完成。

3.1.3.7 my_fdcan 移植文件说明

1. 发送

代码块

```
1  FDCAN_TxHeaderTypeDef TxHeader =
2  {
3      .TxFrameType = FDCAN_DATA_FRAME,
4      .ErrorStateIndicator = FDCAN_ESI_ACTIVE,
5      .BitRateSwitch = FDCAN_BRS_ON,
6      .FDFormat = FDCAN_FD_CAN,
7      .TxEventFifoControl = FDCAN_NO_TX_EVENTS,
8      .MessageMarker = 0,
9  };
10
11 void fdcan_send(FDCAN_HandleTypeDef *fdcanHandle, uint32_t id, uint8_t
    *data, uint16_t size)
12 {
13     TxHeader.Identifier = id;
14
15     if(id > 0x7ff)
16     {
17         TxHeader.IdType = FDCAN_EXTENDED_ID;
18     }
19     else
20     {
```

```

21
22         TxHeader.IdType = FDCAN_STANDARD_ID;
23     }
24     TxHeader.DataLength = get_fdcan_dlc(size);
25     HAL_FDCAN_AddMessageToTxFifoQ(fdcanHandle, &TxHeader, data);
26 }

```

- `FDCAN_TxHeaderTypeDef TxHeader` 是发送的句柄，配置为fdcan数据帧、错误状态指示器、启用比特率切换、使用CAN FD帧格式、不生成发送事件记录到Tx Event FIFO、消息标记设置为0
- `fdcan_send` 是自动识别标准帧和扩展帧并发送CAN FD数据的函数

2. 数据长度字节数与DLC编码值转换

代码块

```

1  uint32_t get_fdcan_dlc(uint16_t size)
2  {
3      uint32_t fdcan_dlc = 0;
4
5      if(size == 0)
6      {
7          fdcan_dlc = FDCAN_DLC_BYTES_0;
8      }
9      else if(size <= 1)
10     {
11         fdcan_dlc = FDCAN_DLC_BYTES_1;
12     }
13     else if(size <= 2)
14     {
15         fdcan_dlc = FDCAN_DLC_BYTES_2;
16     }
17     else if(size <= 3)
18     {
19         fdcan_dlc = FDCAN_DLC_BYTES_3;
20     }
21     else if(size <= 4)
22     {
23         fdcan_dlc = FDCAN_DLC_BYTES_4;
24     }
25     else if(size <= 5)
26     {
27         fdcan_dlc = FDCAN_DLC_BYTES_5;
28     }
29     else if(size <= 6)
30     {

```

```
31         fdcan_dlc = FDCAN_DLC_BYTES_6;
32     }
33     else if(size <= 7)
34     {
35         fdcan_dlc = FDCAN_DLC_BYTES_7;
36     }
37     else if(size <= 8)
38     {
39         fdcan_dlc = FDCAN_DLC_BYTES_8;
40     }
41     else if(size <= 12)
42     {
43         fdcan_dlc = FDCAN_DLC_BYTES_12;
44     }
45     else if(size <= 16)
46     {
47         fdcan_dlc = FDCAN_DLC_BYTES_16;
48     }
49     else if(size <= 20)
50     {
51         fdcan_dlc = FDCAN_DLC_BYTES_20;
52     }
53     else if(size <= 24)
54     {
55         fdcan_dlc = FDCAN_DLC_BYTES_24;
56     }
57     else if(size <= 32)
58     {
59         fdcan_dlc = FDCAN_DLC_BYTES_32;
60     }
61     else if(size <= 48)
62     {
63         fdcan_dlc = FDCAN_DLC_BYTES_48;
64     }
65     else if(size <= 64)
66     {
67         fdcan_dlc = FDCAN_DLC_BYTES_64;
68     }
69     return fdcan_dlc;
70 }
71
72
73 uint16_t get_fdcan_data_size(uint32_t dlc)
74 {
75     uint16_t size = 0;
76
77     switch (dlc)
```

```
78     {
79         case FDCAN_DLC_BYTES_0:
80             size = 0;
81             break;
82         case FDCAN_DLC_BYTES_1:
83             size = 1;
84             break;
85         case FDCAN_DLC_BYTES_2:
86             size = 2;
87             break;
88         case FDCAN_DLC_BYTES_3:
89             size = 3;
90             break;
91         case FDCAN_DLC_BYTES_4:
92             size = 4;
93             break;
94         case FDCAN_DLC_BYTES_5:
95             size = 5;
96             break;
97         case FDCAN_DLC_BYTES_6:
98             size = 6;
99             break;
100        case FDCAN_DLC_BYTES_7:
101            size = 7;
102            break;
103        case FDCAN_DLC_BYTES_8:
104            size = 8;
105            break;
106        case FDCAN_DLC_BYTES_12:
107            size = 12;
108            break;
109        case FDCAN_DLC_BYTES_16:
110            size = 16;
111            break;
112        case FDCAN_DLC_BYTES_20:
113            size = 20;
114            break;
115        case FDCAN_DLC_BYTES_24:
116            size = 24;
117            break;
118        case FDCAN_DLC_BYTES_32:
119            size = 32;
120            break;
121        case FDCAN_DLC_BYTES_48:
122            size = 48;
123            break;
124        case FDCAN_DLC_BYTES_64:
```

```

125         size = 64;
126         break;
127     default:
128         break;
129     }
130
131     return size;
132 }

```

- 这两个函数实现了CAN FD中数据长度字节数与DLC编码值之间的双向转换：

`get_fdcan_dlc` 将实际字节数映射为CAN FD协议的DLC编码，`get_fdcan_data_size` 则将DLC编码反向转换为对应的字节数。

- 使用时直接复制就可以使用

3. fdcan过滤器

代码块

```

1  void fdcan_filter_init(FDCAN_HandleTypeDef *fdcanHandle)
2  {
3      if (HAL_FDCAN_ConfigGlobalFilter(fdcanHandle, FDCAN_ACCEPT_IN_RX_FIFO0,
4          FDCAN_ACCEPT_IN_RX_FIFO0, FDCAN_FILTER_REMOTE, FDCAN_FILTER_REMOTE) != HAL_OK)
5          {
6              Error_Handler();
7          }
8      if (HAL_FDCAN_ActivateNotification(
9          fdcanHandle, FDCAN_IT_RX_FIFO0_NEW_MESSAGE |
10         FDCAN_IT_TX_FIFO_EMPTY, 0) != HAL_OK)
11          {
12              Error_Handler();
13          }
14      HAL_FDCAN_ConfigTxDelayCompensation(fdcanHandle, fdcanHandle->Init.DataPrescaler * fdcanHandle->Init.DataTimeSeg1, 0);
15      HAL_FDCAN_EnableTxDelayCompensation(fdcanHandle);
16      if (HAL_FDCAN_Start(fdcanHandle) != HAL_OK)
17          {
18              Error_Handler();
19          }
20  }

```

- 这是一个全接受过滤器，相当于"接收所有数据帧"。

3.2 配置（第一次使用时必须配置）

此步骤根据您的实际硬件连接进行配置。

3.2.1 修改 `motor.h` 中的宏定义

1. 修改宏定义 `MOTOR_MAX_NUM`，根据单个 CAN 通道需连接的最大电机数量修改。
 - 假设需要使用三个 CAN 通道，其中 CAN1 接 1 个电机，CAN2 接 2 个电机，CAN3 接 5 个电机，宏定义 `MOTOR_MAX_NUM` 修改为 5（以最大使用电机数设置）。

代码块

```
1  #define  MOTOR_MAX_NUM  5
```

2. 修改宏定义 `MOTOR_PORT_NUM`，需要使用个 CAN 通道用于电机通讯就改成多少。
 - 假设 STM32G474 一共有三个 CAN 通道，其中 CAN1 和 CAN3 两个 CAN 通道用于连接电机，宏定义 `MOTOR_PORT_NUM` 修改为 2。

代码块

```
1  #define  MOTOR_PORT_NUM  2  // 通道数量
```

3.2.2 修改 `motor_state_port` 结构体数组（配置电机型号）

1. 在 `motor.c` 文件中，定义了 `motor_state_port[MOTOR_PORT_NUM][MOTOR_MAX_NUM]` 结构体数组，需要根据电机型号进行修改，以进行正确的力矩修正。
2. 假设我们一共要连接四个电机，每个通道连接两个电机，分布如下：

1. `.PORT1`：

`1.4438_30`

`2.5047_36`

2. `PORT2`：

`1.6056_36`

`2.5046_20`

则 `motor_state_port` 配置如下

代码块

```
1  static motor_state_s motor_state_port[MOTOR_PORT_NUM][MOTOR_MAX_NUM] =  // 下标
    + 1 = 电机 ID
2  {
3      {  // CAN 通道 PORT1
4          {  // ID = 1
```

```

5         .model = M4438_30,
6     },
7
8     { // ID = 2
9         .model = M5047_36,
10    },
11 },
12
13 { // CAN 通道 PORT2
14     { // ID = 1
15         .model = M6056_36,
16     },
17
18     { // ID = 2
19         .model = M5046_20,
20     }
21 },
22 };

```

3.2.3 修改 PORT、FDCAN、STATE 的映射关系

1. 在 `motor.c` 中，定义了结构体数组 `port_mapping[MOTOR_PORT_NUM]` 用于定义 PORT、FDCAN、STATE 之间的映射关系。
2. 假设我们用到了 CAN2 和 CAN3 两个通道，我们希望定义
 - CAN2 映射到 PORT1，其状态数据存储在 `motor_state_port[0]`。
 - CAN3 映射到 PORT2，其状态数据存储在 `motor_state_port[1]`。
3. 则对应修改如下：

代码块

```

1  const port_mapping_s port_mapping[MOTOR_PORT_NUM] = // 通道映射表
2  {
3      {
4          .port = PORT1,
5          .fdcan = &hfdcan2,
6          .state = motor_state_port[0],
7      },
8
9      {
10         .port = PORT2,
11         .fdcan = &hfdcan3,
12         .state = motor_state_port[1],
13     },
14 };

```

3.2.4 修改位置速度的单位

- 位置 and 速度单位支持：**转(rev)**、**弧度(rad)**、**或度(°)**，通过 `convert.h` 中的宏定义 `MOTOR_DATA_TYPE_FLAG` 进行切换，默认为**转(rev)**。

1. 如果想使用**转 (rev)** 做为位置速度的单位：

代码块

```
1 #define MOTOR_DATA_TYPE_FLAG TURNS
```

2. 如果想使用**弧度(rad)** 做为位置速度的单位：

代码块

```
1 #define MOTOR_DATA_TYPE_FLAG RADIAN_2PI
```

3. 如果想使用**角度(°)** 做为位置速度的单位：

代码块

```
1 #define MOTOR_DATA_TYPE_FLAG ANGLE_360
```

4. 函数示例

4.1 单电机控制模式 `motor_control.c`

- 此文件内，除电机软重启函数 `motor_set_reset` 外，都带有查询电机状态信息功能（解析函数位于 `motor.c` 文件）。
- 此文件内的所有函数都是调用后立即发送对应的 FDCAN 帧，无软件缓存。
- 此文件内的位置速度单位由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定，详情请看修改位置速度的单位。
- 此文件内的控制方式控制频率不能过高，在仲裁段为 1M，数据段为 5M 的情况下，一个 CAN 总线 1ms 内最多控制 3 个电机，即 1KHz 控制频率下最多控制 3 个电机，如需控制更多电机，需降低控制频率或使用一拖多控制模式（一个控制包括发送指令和接收电机状态信息）。
- 各个参数的单位在代码里的函数注释上都有详细说明。
- `TINT16` 对应的是 `int16` 的数据类型，位置控制只能到 ± 3.2 圈。

4.1.1 DQ 电压模式

说明：

1. 通过给定的 Q 相电压控制电机运行，并让电机返回状态信息。
2. 参数解析：
 - `portx` :CAN 通道选择
 - `type` : 通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
 - `id` : 电机 ID
 - `volt` : Q 相电压，单位：伏特（V）。

3. 函数实现

- `vol_float2int` : 电压值的标度转换。
- `set_dq_volt_float` : `float` 数据类型的底层控制函数。
- `set_dq_volt_int32` : `int32` 数据类型的底层控制函数。
- `set_dq_volt_int16` : `int16` 数据类型的底层控制函数。

代码块

```
1 void motor_set_dq_vlot(port_t portx, const data_type_t type, const uint8_t id,
2   const float volt)
3 {
4     FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
5     const float temp = vol_float2int(volt, type);
6
7     switch(type)
8     {
9         case TFLOAT:
10             set_dq_volt_float(fdcanHandle, id, temp);
11             break;
12         case TINT32:
13             set_dq_volt_int32(fdcanHandle, id, temp);
14             break;
15         case TINT16:
16             set_dq_volt_int16(fdcanHandle, id, temp);
17             break;
18         default:
19             break;
20     }
21 }
```

4.1.2 DQ 电流模式

说明：

1. 通过给定的 Q 相电流控制电机运行，并让电机返回状态信息。
2. 参数解析：
 - `portx` :CAN 通道选择
 - `type` : 通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
 - `id` : 电机 ID
 - `cur` : Q 相电流，单位：安培 (A) 。
3. 函数实现
 - `cur_float2int` : 电流值的标度转换。
 - `set_dq_current_float` : `float` 数据类型的底层控制函数。
 - `set_dq_current_int32` : `int32` 数据类型的底层控制函数。
 - `set_dq_current_int16` : `int16` 数据类型的底层控制函数。

代码块

```
1 void motor_set_dq_current(port_t portx, const data_type_t type, const uint8_t
   id, const float cur)
2 {
3     FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
4     const float temp = cur_float2int(cur, type);
5
6     switch(type)
7     {
8     case TFLOAT:
9         set_dq_current_float(fdcanHandle, id, temp);
10        break;
11    case TINT32:
12        set_dq_current_int32(fdcanHandle, id, temp);
13        break;
14    case TINT16:
15        set_dq_current_int16(fdcanHandle, id, temp);
16        break;
17    default:
18        break;
19    }
20 }
```

4.1.3 位置模式

说明：

1. 电机将以**最大速度**和**最大加速度**运动到指定**目标位置**，并让电机返回状态信息。
2. 参数说明：
 - `portx` :CAN 通道选择
 - `type` : 通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
 - `id` : 电机 ID
 - `pos` : 目标位置，单位可为转 (rev)、弧度 (rad)、或度 (°)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定。

3. 函数实现

- `conv_to_turns` : 将位置的单位转换成转 (rev)
- `pos_float2int` : 位置值的标度转换。
- `set_pos_float` : `float` 数据类型的底层控制函数。
- `set_pos_int32` : `int32` 数据类型的底层控制函数。
- `set_pos_int16` : `int16` 数据类型的底层控制函数。

注意：

1. 该模式下速度与力矩均为最大，运动过程较为激烈。
2. 瞬时电流可能会飙到 5A~10A，若电源响应不够快，或电源电流限制过小，可能导致电机在瞬间获得的电流不足，从而报错。
3. 此模式适用于对响应速度有极端要求的场合，一般不建议使用，如需进行位置控制，推荐使用**梯形控制模式**。

代码块

```
1 void motor_set_pos(port_t portx, const data_type_t type, const uint8_t id,
2   const float pos)
3 {
4     FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
5     const float temp1 = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
6     const float temp2 = pos_float2int(temp1, type);
7
8     switch(type)
9     {
10        case TFLOAT:
11            set_pos_float(fdcanHandle, id, temp2);
12            break;
13        case TINT32:
14            set_pos_int32(fdcanHandle, id, temp2);
```

```

14         break;
15     case TINT16:
16         set_pos_int16(fdcanHandle, id, temp2);
17         break;
18     default:
19         break;
20 }
21 }

```

4.1.4 速度模式

说明：

1. 电机以**最大加速度**加速到指定**目标速度**，并让电机返回状态信息。
2. 参数解析：
 - `portx` :CAN 通道选择
 - `type`：通信协议的数据类型，影响数据的精度和**量程**，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
 - `id`：电机 ID
 - `vel`：目标速度，单位可为转每秒（rps）、弧度每秒（rad/s）、或度每秒（°/s），具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定。
3. 函数实现
 - `conv_to_turns`：将速度的单位转换成转每秒（rps）
 - `vel_float2int`：速度值的标度转换。
 - `set_vel_float`：`float` 数据类型的底层控制函数。
 - `set_vel_int32`：`int32` 数据类型的底层控制函数。
 - `set_vel_int16`：`int16` 数据类型的底层控制函数。

代码块

```

1  void motor_set_vel(port_t portx, const data_type_t type, const uint8_t id,
    const float vel)
2  {
3      FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
4      const float temp1 = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
5      const float temp2 = vel_float2int(temp1, type);
6
7      switch(type)
8      {
9          case TFLOAT:
10             set_vel_float(fdcanHandle, id, temp2);

```

```

11         break;
12     case TINT32:
13         set_vel_int32(fdcanHandle, id, temp2);
14         break;
15     case TINT16:
16         set_vel_int16(fdcanHandle, id, temp2);
17         break;
18     default:
19         break;
20 }
21 }

```

4.1.5 力矩模式

说明：

1. 电机按照设定的目标力矩进行转动，并让电机返回状态信息。
2. 参数解析：
 - `portx` :CAN 通道选择
 - `type`：通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
 - `id`：电机 ID
 - `tqe`：目标力矩，单位：牛·米（N·m）。
3. 函数实现
 - `tqe_adjust`：力矩输出的修正，我们将电机内部的力矩处理在此处处理，用于补偿电机输出的力矩。
 - `tqe_float2int`：力矩值的标度转换。
 - `set_torque_float`：`float` 数据类型的底层控制函数。
 - `set_torque_int32`：`int32` 数据类型的底层控制函数。
 - `set_torque_int16`：`int16` 数据类型的底层控制函数。

代码块

```

1 void motor_set_tqe(port_t portx, const data_type_t type, const uint8_t id,
  const float tqe)
2 {
3     FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
4     const float temp1 = tqe_adjust(tqe, motor_get_model2(portx, id));
5     const float temp2 = tqe_float2int(temp1, type);
6
7     switch(type)

```

```

8      {
9          case TFLOAT:
10             set_torque_float(fdcanHandle, id, temp2);
11             break;
12          case TINT32:
13             set_torque_int32(fdcanHandle, id, temp2);
14             break;
15          case TINT16:
16             set_torque_int16(fdcanHandle, id, temp2);
17             break;
18          default:
19             break;
20      }
21  }

```

4.1.6 位置、速度模式

说明：

1. 电机以目标速度运动至指定的目标位置，并让电机返回状态信息。

2. 参数解析：

- `portx` :CAN 通道选择
- `type` ：通信协议的数据类型，影响数据的精度和量程，支持int16、int32和float
- `id` ：电机 ID
- `pos` ：目标位置，单位可为转（rev）、弧度（rad）、或度（°），具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `vel` ：目标速度，单位可为转每秒（rps）、弧度每秒（rad/s）、或度每秒（°/s），具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定

3. 函数实现

- `conv_to_turns` ：将位置的单位转换成转（rev）、速度的单位转换成转/秒（rps）。
- `pos_float2int` ：位置值的标度转换
- `vel_float2int` ：速度值的标度转换
- `set_pos_vel_tqe_float` ： `float` 数据类型的底层控制函数。
- `set_pos_vel_tqe_int32` ： `int32` 数据类型的底层控制函数。
- `set_pos_vel_tqe_int16` ： `int16` 数据类型的底层控制函数。

代码块

```

1  void motor_set_pos_vel(port_t portx, const data_type_t type, const uint8_t id,
    const float pos, const float vel)

```

```

2  {
3      FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
4      const float pos1 = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
5      const float vel1 = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
6      const float pos2 = pos_float2int(pos1, type);
7      const float vel2 = vel_float2int(vel1, type);
8      switch(type)
9      {
10     case TFLOAT:
11         set_pos_vel_tqe_float(fdcanHandle, id, pos2, vel2, NAN_FLOAT);
12         break;
13     case TINT32:
14         set_pos_vel_tqe_int32(fdcanHandle, id, pos2, vel2, NAN_INT32);
15         break;
16     case TINT16:
17         set_pos_vel_tqe_int16(fdcanHandle, id, pos2, vel2, NAN_INT16);
18         break;
19     default:
20         break;
21     }
22 }

```

4.1.7 位置、速度、最大力矩模式

说明：

1. 电机以目标速度运动至指定的目标位置，同时限制最大输出力矩，并让电机返回状态信息。
2. 参数解析：
 - `portx` :CAN 通道选择
 - `type` ：通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
 - `id` ：电机 ID
 - `pos` ：目标位置，单位可为转（rev）、弧度（rad）、或度（°），具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
 - `vel` ：目标速度，单位可为转每秒（rps）、弧度每秒（rad/s）、或度每秒（°/s），具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
 - `tqe` ：目标力矩，单位：牛·米（N·m）。
3. 函数实现
 - `conv_to_turns` ：将位置的单位转换成转(rev)、速度的单位转换成转/秒（rps）。

- `tqe_adjust`：力矩输出的修正，我们将电机内部的力矩处理在此处处理，用于补偿电机输出的力矩。
- `pos_float2int`：位置值的标度转换。
- `vel_float2int`：速度值的标度转换。
- `tqe_float2int`：力矩值的标度转换。
- `set_pos_vel_tqe_float`：`float` 数据类型的底层控制函数。
- `set_pos_vel_tqe_int32`：`int32` 数据类型的底层控制函数。
- `set_pos_vel_tqe_int16`：`int16` 数据类型的底层控制函数。

注意：

1. 该模式对输出力矩有限制，若设置的最大力矩过小，电机可能无法达到目标速度。

代码块

```

1 void motor_set_pos_vel_MAXtqe(port_t portx, const data_type_t type, const
  uint8_t id,
2                                     const float pos, const float vel, const float
  tqe)
3 {
4     FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
5     const float pos1 = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
6     const float vel1 = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
7     const float tqe1 = tqe_adjust(tqe, motor_get_model2(portx, id));
8     const float pos2 = pos_float2int(pos1, type);
9     const float vel2 = vel_float2int(vel1, type);
10    const float tqe2 = tqe_float2int(tqe1, type);
11    switch(type)
12    {
13        case TFLOAT:
14            set_pos_vel_tqe_float(fdcanHandle, id, pos2, vel2, tqe2);
15            break;
16        case TINT32:
17            set_pos_vel_tqe_int32(fdcanHandle, id, pos2, vel2, tqe2);
18            break;
19        case TINT16:
20            set_pos_vel_tqe_int16(fdcanHandle, id, pos2, vel2, tqe2);
21            break;
22        default:
23            break;
24    }
25 }
```

4.1.8 位置、速度、加速度模式（梯形控制）

说明：

1. 电机按照**恒定加速度**运动，实现先加速 → 匀速 → 减速的梯形速度控制，并让电机返回状态信息。

2. 参数解析：

- `portx` :CAN 通道选择
- `type` : 通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
- `id` : 电机 ID
- `pos` : 目标位置，单位可为转 (rev)、弧度 (rad)、或度 (°)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `vel` : 目标速度，单位可为转每秒 (rps)、弧度每秒 (rad/s)、或度每秒 (°/s)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `acc` : 加速度，单位可为转每秒平方 (rev/s²)、弧度每秒平方 (rad/s²)、或度每秒平方 (°/s²)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定

3. 函数实现

- `conv_to_turns` : 将位置的单位转换成转(rev)、速度的单位转换成转/秒 (rps)、加速度的单位转换成每秒平方 (rps²)
- `pos_float2int` : 位置值的标度转换。
- `vel_float2int` : 速度值的标度转换。
- `acc_float2int` : 加速度值的标度转换。
- `set_pos_velmax_acc_float` : `float` 数据类型的底层控制函数。
- `set_pos_velmax_acc_int32` : `int32` 数据类型的底层控制函数。
- `set_pos_velmax_acc_int16` : `int16` 数据类型的底层控制函数。

代码块

```
1 void motor_set_pos_velmax_acc(port_t portx, const data_type_t type, const
  uint8_t id, const float pos, const float vel, const float acc)
2 {
3     FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
4     const float pos1 = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
5     const float vel1 = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
6     const float acc1 = conv_to_turns(acc, MOTOR_DATA_TYPE_FLAG);
7     const float pos2 = pos_float2int(pos1, type);
8     const float vel2 = vel_float2int(vel1, type);
9     const float acc2 = acc_float2int(acc1, type);
```

```

10     switch(type)
11     {
12     case TFLOAT:
13         set_pos_velmax_acc_float(fdcanHandle, id, pos2, vel2, acc2);
14         break;
15     case TINT32:
16         set_pos_velmax_acc_int32(fdcanHandle, id, pos2, vel2, acc2);
17         break;
18     case TINT16:
19         set_pos_velmax_acc_int16(fdcanHandle, id, pos2, vel2, acc2);
20         break;
21     default:
22         break;
23     }
24 }

```

4.1.9 速度、加速度模式

说明：

1. 以目标加速度加速到目标速度，并让电机返回状态信息。

2. 参数解析：

- `portx` :CAN 通道选择
- `type` : 通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
- `id` : 电机 ID
- `vel` : 目标速度，单位可为转每秒 (rps)、弧度每秒 (rad/s)、或度每秒 (°/s)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `acc` : 加速度，单位可为转每秒平方 (rev/s²)、弧度每秒平方 (rad/s²)、或度每秒平方 (°/s²)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定

3. 函数实现

- `conv_to_turns` : 将位置的单位转换成转(rev)、速度的单位转换成转/秒 (rps)、加速度的单位转换成每秒平方 (rps²)
- `vel_float2int` : 速度值的标度转换。
- `acc_float2int` : 加速度值的标度转换。
- `set_vel_acc_float` : `float` 数据类型的底层控制函数。
- `set_vel_acc_int32` : `int32` 数据类型的底层控制函数。
- `set_vel_acc_int16` : `int16` 数据类型的底层控制函数。

```

代码块
void motor_set_vel_acc(port_t portx, const data_type_t type, const uint8_t id,
    const float vel, const float acc)
2  {
3      FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
4      const float vel1 = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
5      const float acc1 = conv_to_turns(acc, MOTOR_DATA_TYPE_FLAG);
6      const float vel2 = vel_float2int(vel1, type);
7      const float acc2 = acc_float2int(acc1, type);
8      switch(type)
9      {
10     case TFLOAT:
11         set_vel_acc_float(fdcanHandle, id, vel2, acc2);
12         break;
13     case TINT32:
14         set_vel_acc_int32(fdcanHandle, id, vel2, acc2);
15         break;
16     case TINT16:
17         set_vel_acc_int16(fdcanHandle, id, vel2, acc2);
18         break;
19     default:
20         break;
21     }
22 }

```

4.1.10 运控模式（MIT模式）

说明：

1. 电机输出力矩的计算公式为：

代码块

```

1  输出力矩 = 位置偏差 * kp + 速度偏差 * kd + 前馈力矩

```

2. 模式用法说明

- a. 当 $k_p=0$, $k_d=0$ 时，给定 tqe 即可实现给定扭矩输出。在该情况下，电机将持续输出一个恒定力矩。
- b. 当 $k_p=0$, $k_d \neq 0$ 时，给定 vel 即可实现匀速转动。匀速转动过程中存在速度静差，另外 k_d 不宜过大， k_d 过大时会引起震荡。
- c. 当 $k_p \neq 0$, $k_d=0$ 时，会引起震荡。即对位置进行控制时， k_d 不能赋 0，否则会造成电机震荡，甚至失控。
- d. 当 $k_p \neq 0$, $k_d \neq 0$ 时，有多种情况，这里下面简单介绍两种情况。

- 当目标位置 `pos` 为常量，目标速度 `vel` 为0时，可实现定点控制，系统会使实际位置趋近于 `pos`，实际速度趋近于 0，最终静止在目标位置。
- 当 `pos` 是随时间变化的连续可导函数时，同时 `vel` 是 `pos` 的导数，可实现位置跟踪和速度跟踪，即按照期望速度旋转期望角度。

3. 参数解析：

- `portx` :CAN 通道选择
- `type` : 通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
- `id` : 电机 ID
- `pos` : 目标位置，单位可为转 (rev)、弧度 (rad)、或度 (°)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `vel` : 目标速度，单位可为转每秒 (rps)、弧度每秒 (rad/s)、或度每秒 (°/s)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `tqe` : 前馈力矩，单位：牛·米 (N·m)。
- `kp` : 位置比例系数。
- `kd` : 速度比例系数。

4. 函数实现

- `conv_to_turns` : 将位置的单位转换成转(rev)、速度的单位转换成转/秒 (rps)。
- `conv_from_turns` : 将 `kp`、`kd` 转换成对应使用单位的数据，例如使用了弧度，那么通过这个函数就可以将 `kp`、`kd` 的单位也转换成弧度单位。
- `tqe_adjust` : 力矩输出的修正，我们将电机内部的力矩处理在此处处理，用于补偿电机输出的力矩。
- `pos_float2int` : 位置值的标度转换。
- `vel_float2int` : 速度值的标度转换。
- `tqe_float2int` : 力矩值的标度转换。
- `pid_float2int` : `kp`、`kd` 值到电机数据值的标度转换，根据使用的数据类型将力矩值转换为电机控制所需的整数格式。
- `set_pos_vel_tqe_kp_kd_` : 根据数据类型自动选择对应的底层控制函数。

代码块

```
1 void motor_set_pos_vel_tqe_kp_kd_2(port_t portx, const data_type_t type, const
  uint8_t id,
2                                     const float pos, const float vel, const
  float tqe, const float kp, const float kd)
```

```

3  {
4      FDCAN_HandleTypeDef *fdcanHandle = motor_get_fdcan_pointer(portx);
5      const motor_type_t model = motor_get_model2(portx, id);
6
7      /* 单位转换成转 */
8      const float pos_turns = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
9      const float vel_turns = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
10     const float kp_turns = conv_from_turns(kp, MOTOR_DATA_TYPE_FLAG);
11     const float kd_turns = conv_from_turns(kd, MOTOR_DATA_TYPE_FLAG);
12
13     /* 力矩修正 */
14     const float tqe_val_adjust = tqe_adjust(tqe, model);
15     const float kp_val_adjust = pid_adjust(kp_turns, model);
16     const float kd_val_adjust = pid_adjust(kd_turns, model);
17
18     /* float -> int */
19     const float pos_int = pos_float2int(pos_turns, type);
20     const float vel_int = vel_float2int(vel_turns, type);
21     const float tqe_int = tqe_float2int(tqe_val_adjust, type);
22     const float kp_int = pid_float2int(kp_val_adjust, type);
23     const float kd_int = pid_float2int(kd_val_adjust, type);
24
25     switch(type)
26     {
27     case TFLOAT:
28         set_pos_vel_tqe_kp_kd_float_2(fdcanHandle, id, pos_int, vel_int,
29         tqe_int, kp_int, kd_int);
30         break;
31     case TINT32:
32         set_pos_vel_tqe_kp_kd_int32_2(fdcanHandle, id, pos_int, vel_int,
33         tqe_int, kp_int, kd_int);
34         break;
35     case TINT16:
36         set_pos_vel_tqe_kp_kd_int16_2(fdcanHandle, id, pos_int, vel_int,
37         tqe_int, kp_int, kd_int);
38         break;
39     default:
40         break;
41     }
42 }

```

注意：

1. 真正的运控模式，和 `motor_set_pos_vel_tqe_kp_kd` 区别是：
 - `motor_set_pos_vel_tqe_kp_kd` 的位置是不断积分得到的，在低频控制下容易出现意外情况。

- `motor_set_pos_vel_tqe_kp_kd_2` 则避免了此问题，控制更稳定。

2. 该需要设置合适的 `kp` 和 `kd`，否则控制效果可能较差。

代码块

```
1  /* 不建议使用 */
2  void motor_set_pos_vel_tqe_kp_kd(port_t portx, const data_type_t type, const
    uint8_t id,
3  const float pos, const float vel, const float tqe, const float kp, const float
    kd);
4
5  /* 建议使用 */
6  void motor_set_pos_vel_tqe_kp_kd_2(port_t portx, const data_type_t type, const
    uint8_t id,
7  const float pos, const float vel, const float tqe, const float kp, const float
    kd);
```

4.1.11 查询电机状态信息

说明：

1. 发送查询电机状态信息的指令。
2. 电机返回的状态信息包括，模式、错误码、位置、速度、力矩。
3. 电机状态信息解析在 `motor.c` 的 `motor_process_state` 函数。
4. 参数解析：
 - `portx` :CAN 通道选择
 - `type`：通信协议的数据类型，影响数据的精度和量程，`TFLOAT`、`TINT32`、`TINT16` 对应 `float`、`int32`、`int16`
 - `id`：电机 ID

代码块

```
1  void motor_get_state_send(port_t portx, const data_type_t type, const uint8_t
    id);
```

4.1.12 查询电机固件版本

说明：

1. 发送查询电机固件版本号指令。
2. 电机固件版本解析在 `motor.c` 的 `motor_process_state` 函数。
3. 参数解析：

- `portx`:CAN 通道选择
- `id`: 电机 ID

代码块

```
1 void motor_get_version(port_t portx, const uint8_t id);
```

4.1.13 停止模式

说明:

1. 电机进入停止模式，电机三相都悬空，使电机可以自由转动。
2. 参数解析：
 - `portx`:CAN 通道选择
 - `id`: 电机 ID

代码块

```
1 void motor_set_stop(port_t portx, const uint8_t id);
```

4.1.14 刹车模式

说明:

1. 将电机所有相短接到地，实现“阻尼刹车”效果。
2. 刹车阻力与电机转速成正相关。
3. 参数解析：
 - `portx`:CAN 通道选择
 - `id`: 电机 ID

代码块

```
1 void motor_set_brake(port_t portx, const uint8_t id);
```

4.1.15 电机软重启

说明:

1. 对电机执行软件重启。，重启后进入停止模式。
2. 不会有任何反馈。
3. 参数解析:

- `portx`:CAN 通道选择
- `id`: 电机 ID

代码块

```
1 void motor_set_reset(port_t portx, const uint8_t id);
```

4.2 一拖多控制模式 `motor_many.c`

- 此文件内，只有 `motor_many_send` 是用于发送指令的，其他的函数都是向缓存写入指令。
- 一拖多模式下，同一条 CAN 通道上的电机模式是相同的。
- 一拖多模式大致原理，发送一条通用的 FDCAN 帧，帧上不同的字节控制不同电机，其中由 ID 确定电机模式，详细说明请看 02-fdcan 协议解析。
- 一拖多模式下，在仲裁段为 1M，数据段为 5M 的情况下，一个 CAN 总线 1ms 内最多控制 10 个电机，即 1KHz 控制频率下最多控制 10 个电机，如需控制更多电机，需降低控制频率。
- 一拖多模式使用 `int16` 的数据类型，位置控制只能到 ± 3.2 圈。
- 一拖多模式使用说明：

代码块

```
1  /* 写入缓存 */
2  motor_many_vel(PORT1, 1, 0.1);
3  motor_many_vel(PORT1, 2, 0.1);
4  motor_many_vel(PORT1, 3, 0.1);
5
6  /* 发送指令 */
7  motor_many_send(PORT1);
```

4.2.1 DQ 电压模式

说明：

1. 通过给定的 Q 相电压控制电机运行，并让电机返回状态信息。

2. 参数解析：

- `portx`:CAN 通道选择
- `id`: 电机 ID
- `volt`: Q 相电压，单位：伏特（V）。

3. 函数实现

- `motor_get_many_pointer`:根据 `portx` 选择对应通道的缓冲区

- `vol_float2int`：电压值的标度转换。
- `p_many_data`：参数暂存在 `p_many_data` 缓冲区中，一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```
1 void motor_many_dq_volt(port_t portx, const uint8_t id, const float vol)
2 {
3     p_many_data_s p_many_data = motor_get_many_pointer(portx);
4     const int16_t vol_int16 = vol_float2int(vol, TINT16);
5     const uint16_t index = id - 1;
6
7     if (p_many_data->mode != MODE_VOLTAGE)
8     {
9         p_many_data->mode = MODE_VOLTAGE;
10        for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)
11        {
12            p_many_data->data16[i] = NAN_INT16;
13        }
14    }
15
16    p_many_data->voltage[index] = vol_int16;
17 }
```

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

4.2.2 DQ 电流模式

说明：

1. 通过给定的 Q 相电流控制电机运行，并让电机返回状态信息。

2. 参数解析：

- `portx`：CAN 通道选择
- `id`：电机 ID
- `cur`：Q 相电流，单位：安培（A）。

3. 函数实现

- `motor_get_many_pointer`：根据 `portx` 选择对应通道的缓冲区
- `cur_float2int`：电流值的标度转换。

- `p_many_data`：参数暂存在 `p_many_data` 缓冲区中，一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```
1 void motor_many_dq_current(port_t portx, const uint8_t id, const float cur)
2 {
3     p_many_data_s p_many_data = motor_get_many_pointer(portx);
4     const int16_t cur_int16 = cur_float2int(cur, TINT16);
5     const uint16_t index = id - 1;
6
7     if (p_many_data->mode != MODE_CURRENT)
8     {
9         p_many_data->mode = MODE_CURRENT;
10        for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)
11        {
12            p_many_data->data16[i] = NAN_INT16;
13        }
14    }
15
16    p_many_data->current[index] = cur_int16;
17 }
```

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

4.2.3 位置模式

说明：

1. 电机将以最大速度和最大加速度运动到指定目标位置，并让电机返回状态信息。
2. 参数解析：
 - `portx`：CAN 通道选择
 - `id`：电机 ID
 - `pos`：目标位置，单位可为转 (rev)、弧度 (rad)、或度 (°)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
3. 函数实现
 - `motor_get_many_pointer`：根据 `portx` 选择对应通道的缓冲区
 - `conv_to_turns`：将位置、速度的单位转换成转(rev)。
 - `pos_float2int`：位置值的标度转换。

- `p_many_data`：参数暂存在 `p_many_data` 缓冲区中，一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```
1 void motor_many_pos(port_t portx, const uint8_t id, const float pos)
2 {
3     p_many_data_s p_many_data = motor_get_many_pointer(portx);
4     const float pos_turns = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
5     const int16_t pos_int16 = pos_float2int(pos_turns, TINT16);
6     const uint16_t index = id - 1;
7
8     if (p_many_data->mode != MODE_POSITION)
9     {
10         p_many_data->mode = MODE_POSITION;
11         for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)
12         {
13             p_many_data->data16[i] = NAN_INT16;
14         }
15     }
16
17     p_many_data->position[index] = pos_int16;
18 }
```

注意：

1. 该模式下速度与力矩均为最大，运动过程较为激烈。
2. 瞬时电流可能会飙到 5A~10A，若电源响应不够快，或电源电流限制过小，可能导致电机在瞬间获得的电流不足，从而报错。
3. 此模式适用于对响应速度有极端要求的场合，一般不建议使用，如需进行位置控制，推荐使用**梯形控制模式**。
4. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

4.2.4 速度模式

说明：

1. 电机以最大加速度加速到指定目标速度，并让电机返回状态信息。
2. 参数解析：
 - `portx` :CAN 通道选择
 - `id`：电机 ID

- `vel`：目标速度，单位可为转每秒（rps）、弧度每秒（rad/s）、或度每秒（°/s），具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定

3. 函数实现

- `motor_get_many_pointer`：根据 `portx` 选择对应通道的缓冲区
- `conv_to_turns`：将位置、速度的单位转换成转(rev)。
- `vel_float2int`：速度值的标度转换。
- `p_many_data`：参数暂存在 `p_many_data` 缓冲区中，一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```
1 void motor_many_vel(port_t portx, const uint8_t id, const float vel)
2 {
3     p_many_data_s p_many_data = motor_get_many_pointer(portx);
4     const float vel_turns = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
5     const int16_t vel_int16 = vel_float2int(vel_turns, TINT16);
6     const uint16_t index = id - 1;
7
8     if (p_many_data->mode != MODE_VELOCITY)
9     {
10         p_many_data->mode = MODE_VELOCITY;
11         for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)
12         {
13             p_many_data->data16[i] = NAN_INT16;
14         }
15     }
16
17     p_many_data->velocity[index] = vel_int16;
18 }
```

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

4.2.5 力矩模式

说明：

1. 电机按照设定的目标力矩进行转动，并让电机返回状态信息。
2. 参数解析：
 - `portx`：CAN 通道选择

- `id`：电机 ID
- `tqe`：目标力矩，单位：牛·米（N·m）。

3. 函数实现

- `motor_get_many_pointer`：根据 `portx` 选择对应通道的缓冲区
- `tqe_adjust`：力矩输出的修正，我们将电机内部的力矩处理在此处处理，用于补偿电机输出的力矩。
- `tqe_float2int`：力矩值的标度转换。
- `p_many_data`：参数暂存在 `p_many_data` 缓冲区中，一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```
1 void motor_many_tqe(port_t portx, const uint8_t id, const float tqe)
2 {
3     p_many_data_s p_many_data = motor_get_many_pointer(portx);
4     const float tqe_float = tqe_adjust(tqe, motor_get_model2(portx, id));
5     const int16_t tqe_int16 = tqe_float2int(tqe_float, TINT16);
6     const uint16_t index = id - 1;
7
8     if (p_many_data->mode != MODE_TORQUE)
9     {
10         p_many_data->mode = MODE_TORQUE;
11         for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)
12         {
13             p_many_data->torque[i] = NAN_INT16;
14         }
15     }
16
17     p_many_data->torque[index] = tqe_int16;
18 }
```

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

4.2.6 位置、速度模式

说明：

1. 电机以目标速度运动至指定的目标位置，并让电机返回状态信息。
2. 参数解析：

- `portx`: CAN 通道选择
- `id`: 电机 ID
- `pos`: 目标位置, 单位可为转 (rev)、弧度 (rad)、或度 (°), 具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `vel`: 目标速度, 单位可为转每秒 (rps)、弧度每秒 (rad/s)、或度每秒 (°/s), 具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定

3. 函数实现

- `motor_get_many_pointer`: 根据 `portx` 选择对应通道的缓冲区
- `conv_to_turns`: 将位置、速度的单位转换成转 (rev)。
- `pos_float2int`: 位置值的标度转换。
- `vel_float2int`: 速度值的标度转换。
- `p_many_data`: 参数暂存在 `p_many_data` 缓冲区中, 一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```

1  void motor_many_pos_vel(port_t portx, const uint8_t id, const float pos, const
    float vel)
2  {
3      p_many_data_s p_many_data = motor_get_many_pointer(portx);
4
5      const float pos_turns = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
6      const float vel_turns = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
7
8      const int16_t pos_int16 = pos_float2int(pos_turns, TINT16);
9      const int16_t vel_int16 = vel_float2int(vel_turns, TINT16);
10     const uint16_t index = id - 1;
11
12     if (p_many_data->mode != MODE_POS_VEL_TQE)
13     {
14         p_many_data->mode = MODE_POS_VEL_TQE;
15         for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)
16         {
17             p_many_data->data16[i] = NAN_INT16;
18         }
19     }
20
21     p_many_data->pos_vel_tqe[index].pos = pos_int16;
22     p_many_data->pos_vel_tqe[index].vel = vel_int16;
23     p_many_data->pos_vel_tqe[index].tqe = NAN_INT16;
24 }
```

注意：

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

4.2.7 位置、速度、最大力矩模式

说明：

1. 电机以目标速度运动至指定的目标位置，同时限制最大输出力矩，并让电机返回状态信息。

2. 参数解析：

- `portx` :CAN 通道选择
- `id` : 电机 ID
- `pos` : 目标位置，单位可为转 (rev)、弧度 (rad)、或度 (°)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `vel` : 目标速度，单位可为转每秒 (rps)、弧度每秒 (rad/s)、或度每秒 (°/s)，具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `tqe` : 目标力矩，单位：牛·米 (N·m)。

3. 函数实现

- `motor_get_many_pointer` :根据 `portx` 选择对应通道的缓冲区
- `conv_to_turns` : 将位置、速度的单位转换成转(rev)。
- `tqe_adjust` : 力矩输出的修正，我们将电机内部的力矩处理在此处处理，用于补偿电机输出的力矩。
- `pos_float2int` : 位置值的标度转换。
- `vel_float2int` : 速度值的标度转换。
- `tqe_float2int` : 力矩值的标度转换。
- `p_many_data` : 参数暂存在 `p_many_data` 缓冲区中，一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```
1 void motor_many_pos_vel_MAXtqe(port_t portx, const uint8_t id, const float
  pos, const float vel, const float tqe)
2 {
3     p_many_data_s p_many_data = motor_get_many_pointer(portx);
4
5
6     const float pos_turns = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
7     const float vel_turns = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
```

```

8      const float tqe_float = tqe_adjust(tqe, motor_get_model2(portx, id));
9
10     const int16_t pos_int16 = pos_float2int(pos_turns, TINT16);
11     const int16_t vel_int16 = vel_float2int(vel_turns, TINT16);
12     const int16_t tqe_int16 = tqe_float2int(tqe_float, TINT16);
13     const uint16_t index = id - 1;
14
15     if (p_many_data->mode != MODE_POS_VEL_TQE)
16     {
17         p_many_data->mode = MODE_POS_VEL_TQE;
18         for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)
19         {
20             p_many_data->data16[i] = NAN_INT16;
21         }
22     }
23
24     p_many_data->pos_vel_tqe[index].pos = pos_int16;
25     p_many_data->pos_vel_tqe[index].vel = vel_int16;
26     p_many_data->pos_vel_tqe[index].tqe = tqe_int16;
27 }

```

注意：

1. 该模式对输出力矩有限制，若设置的最大力矩过小，电机可能无法达到目标速度。
2. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

4.2.8 位置、速度、加速度模式（梯形控制）

说明：

1. 电机按照恒定加速度运动，实现先加速 → 匀速 → 减速的梯形速度控制，并让电机返回状态信息。
2. 参数解析：
 - `portx`：CAN 通道选择
 - `id`：电机 ID
 - `pos`：目标位置，单位可为转（rev）、弧度（rad）、或度（°），具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
 - `vel`：目标速度，单位可为转每秒（rps）、弧度每秒（rad/s）、或度每秒（°/s），具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
 - `acc`：加速度，单位：转每秒平方（rps²）。
3. 函数实现

- `motor_get_many_pointer`: 根据 `portx` 选择对应通道的缓冲区
- `conv_to_turns`: 将位置、速度的单位转换成转(rev)。
- `pos_float2int`: 位置值的标度转换。
- `vel_float2int`: 速度值的标度转换。
- `acc_float2int`: 加速度值的标度转换。
- `p_many_data`: 参数暂存在 `p_many_data` 缓冲区中，一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```

1  void motor_many_pos_vel_acc(port_t portx, const uint8_t id, const float pos,
   const float vel, const float acc)
2  {
3      p_many_data_s p_many_data = motor_get_many_pointer(portx);
4
5      const float pos_turns = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
6      const float vel_turns = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
7      const float acc_turns = conv_to_turns(acc, MOTOR_DATA_TYPE_FLAG);
8
9      const int16_t pos_int16 = pos_float2int(pos_turns, TINT16);
10     const int16_t vel_int16 = vel_float2int(vel_turns, TINT16);
11     const int16_t acc_int16 = acc_float2int(acc_turns, TINT16);
12     const uint16_t index = id - 1;
13
14     if (p_many_data->mode != MODE_POS_VEL_ACC)
15     {
16         p_many_data->mode = MODE_POS_VEL_ACC;
17         for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)
18         {
19             p_many_data->data16[i] = NAN_INT16;
20         }
21     }
22
23     p_many_data->pos_vel_tqe[index].pos = pos_int16;
24     p_many_data->pos_vel_tqe[index].vel = vel_int16;
25     p_many_data->pos_vel_tqe[index].tqe = acc_int16;
26 }
```

注意:

1. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

4.2.9 运控模式（MIT模式）

说明：

1. 电机输出力矩的计算公式为：

代码块

```
1  输出力矩 = 位置偏差 * kp + 速度偏差 * kd + 前馈力矩
```

2. 模式用法说明

- a. 当 $kp=0$, $kd=0$ 时, 给定 tqe 即可实现给定扭矩输出。在该情况下, 电机会持续输出一个恒定力矩。
- b. 当 $kp=0$, $kd \neq 0$ 时, 给定 vel 即可实现匀速转动。匀速转动过程中存在速度静差, 另外 kd 不宜过大, kd 过大时会引起震荡。
- c. 当 $kp \neq 0$, $kd=0$ 时, 会引起震荡。即对位置进行控制时, kd 不能赋 0, 否则会造成电机震荡, 甚至失控。
- d. 当 $kp \neq 0$, $kd \neq 0$ 时, 有多种情况, 这里下面简单介绍两种情况。
 - 当目标位置 pos 为常量, 目标速度 vel 为 0 时, 可实现定点控制, 系统会使实际位置趋近于 pos , 实际速度趋近于 0, 最终静止在目标位置。
 - 当 pos 是随时间变化的连续可导函数时, 同时 vel 是 pos 的导数, 可实现位置跟踪和速度跟踪, 即按照期望速度旋转期望角度。

3. 参数解析：

- `portx` :CAN 通道选择
- `id` : 电机 ID
- `pos` : 目标位置, 单位可为转 (rev)、弧度 (rad)、或度 (°), 具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `vel` : 目标速度, 单位可为转每秒 (rps)、弧度每秒 (rad/s)、或度每秒 (°/s), 具体由宏定义 `MOTOR_DATA_TYPE_FLAG` 决定
- `tqe` : 前馈力矩, 单位: 牛·米 (N·m)。
- `kp` : 位置比例系数。
- `kd` : 速度比例系数。

4. 函数实现

- `motor_get_many_pointer` :根据 `portx` 选择对应通道的缓冲区
- `conv_to_turns` : 将位置、速度的单位转换成转(rev)。

- `conv_from_turns`：将 `kp`、`kd` 转换成对应使用单位的数据，例如使用了弧度，那么通过这个函数就可以将 `kp`、`kd` 的单位也转换成弧度单位。
- `tqe_adjust`：力矩输出的修正，我们将电机内部的力矩处理在此处处理，用于补偿电机输出的力矩。
- `pos_float2int`：位置值的标度转换。
- `vel_float2int`：速度值的标度转换。
- `tqe_float2int`：力矩值的标度转换。
- `pid_float2int`：`kp`、`kd` 值的标度转换。
- `p_many_data`：参数暂存在 `p_many_data` 缓冲区中，一拖多模式是通过单次通信将所有电机控制指令统一发送

代码块

```

1  void motor_many_pos_vel_tqe_kp_kd_2(port_t portx, const uint8_t id, const
    float pos, const float vel, const float tqe, const float kp, const float kd)
2  {
3      p_many_data_s p_many_data = motor_get_many_pointer(portx);
4      const motor_type_t model = motor_get_model2(portx, id);
5
6      /* 单位转换成转 */
7      const float pos_turns = conv_to_turns(pos, MOTOR_DATA_TYPE_FLAG);
8      const float vel_turns = conv_to_turns(vel, MOTOR_DATA_TYPE_FLAG);
9      const float kp_turns = conv_from_turns(kp, MOTOR_DATA_TYPE_FLAG);
10     const float kd_turns = conv_from_turns(kd, MOTOR_DATA_TYPE_FLAG);
11
12     /* 力矩修正 */
13     const float tqe_val_adjust = tqe_adjust(tqe, model);
14     const float kp_val_adjust = pid_adjust(kp_turns, model);
15     const float kd_val_adjust = pid_adjust(kd_turns, model);
16
17     /* float -> int */
18     const float pos_int16 = pos_float2int(pos_turns, TINT16);
19     const float vel_int16 = vel_float2int(vel_turns, TINT16);
20     const float tqe_int16 = tqe_float2int(tqe_val_adjust, TINT16);
21     const float kp_int16 = pid_float2int(kp_val_adjust, TINT16);
22     const float kd_int16 = pid_float2int(kd_val_adjust, TINT16);
23
24     const uint16_t index = id - 1;
25
26     if (p_many_data->mode != MODE_POS_VEL_TQE_KP_KD_2)
27     {
28         p_many_data->mode = MODE_POS_VEL_TQE_KP_KD_2;
29         for (int i = 0; i < MANY_DATA_BUF_MAX_LEN / sizeof(int16_t); i++)

```

```

30      {
31          p_many_data->data16[i] = NAN_INT16;
32      }
33  }
34
35  p_many_data->pos_vel_tqe_kp_kd[index].pos = pos_int16;
36  p_many_data->pos_vel_tqe_kp_kd[index].vel = vel_int16;
37  p_many_data->pos_vel_tqe_kp_kd[index].tqe = tqe_int16;
38  p_many_data->pos_vel_tqe_kp_kd[index].kp = kp_int16;
39  p_many_data->pos_vel_tqe_kp_kd[index].kd = kd_int16;
40  }

```

注意：

5. 真正的运控模式，和 `motor_many_pos_vel_tqe_kp_kd` 区别是：
 - `motor_many_pos_vel_tqe_kp_kd` 的位置是不断积分得到的，在低频控制下容易出现意外情况。
 - `motor_many_pos_vel_tqe_kp_kd_2` 则避免了此问题，控制更稳定。
6. 该需要设置合适的 `kp` 和 `kd`，否则控制效果可能较差。
7. 此函数仅将指令写入缓冲区，不会立即发送，需要调用 `motor_many_send` 才会生效。

代码块

```

1  /* 不建议使用 */
2  void motor_many_pos_vel_tqe_kp_kd(port_t portx, const uint8_t id,
3  const float pos, const float vel, const float tqe, const float kp, const float
4  kd);
5  /* 建议使用 */
6  void motor_many_pos_vel_tqe_kp_kd_2(port_t portx, const uint8_t id,
7  const float pos, const float vel, const float tqe, const float kp, const float
8  kd);

```

4.3 电机设置 `motor_config`

- 此文件内的函数都是用于电机设置相关。

4.3.1 重置电机零位

说明：

1. 功能：
 - 将电机当前位置设置成零位。

- 重置一个电机的零位需要要 300ms 以上，且有失败可能。
- 返回值：
 - 0-成功
 - 1-失败，表示重置电机零位失败
 - 2-失败，表示写入设置失败

2. 参数解析：

- `portx` :CAN 通道选择
- `id` : 电机 ID

代码块

```
1 uint8_t motor_pos_reset(port_t portx, const uint8_t id);
```

4.3.2 保存电机设置

说明：

1. 功能

- 保存电机设置。
- 一般配合其他指令运行。
- 返回值：
 - 0-成功
 - 1-失败，表示写入设置失败

2. 参数解析：

- `portx` :CAN 通道，`PORT1`、`PORT2`、`PORT3` 对应通道 1、2、3
- `id` : 电机 ID

代码块

```
1 uint8_t motor_conf_write(port_t portx, const uint8_t id);
```

4.4 电机配置和协议解析 `motor`

- 此文件中定义了两个重要的结构体数组：详见 3.2配置
 - `motor_state_port[MOTOR_PORT_NUM][MOTOR_MAX_NUM]`：用于配置电机型号和储存电机返回的信息。

- `port_maping[MOTOR_PORT_NUM]`：用于配置 PORT、FDCAN 和 STATE 的映射关系。

4.4.1 电机信息解析

说明：

- 解析所有 CAN 通道 FIFO 中的电机状态数据。
- `HAL_FDCAN_GetRxMessage` 是从指定的FDCAN接收FIFO中读取接收到的CAN报文。
- 此函数中调用 `motor_process_state` 进行解析。
- 此函数需要放在主循环中不断调用，也可根据需求定时调用。

代码块

```
1  void motor_process_state_all()
2  {
3      for (int i = 0; i < MOTOR_MAX_NUM; i++)
4      {
5          while (HAL_FDCAN_GetRxMessage(port_maping[i].fdcan, FDCAN_RX_FIFO0,
6          &fdcan_rx_header, fdcan_rdata) == HAL_OK)
7          {
8              if (fdcan_rx_header.DataLength != 0)
9              {
10                 const uint16_t len =
11                 get_fdcan_data_size(fdcan_rx_header.DataLength);
12                 motor_process_state(port_maping[i].fdcan,
13                 fdcan_rx_header.Identifier >> 8, fdcan_rdata, len);
14             }
15         }
16     }
17 }
```